ECE449 Computer architecture Project explain 2

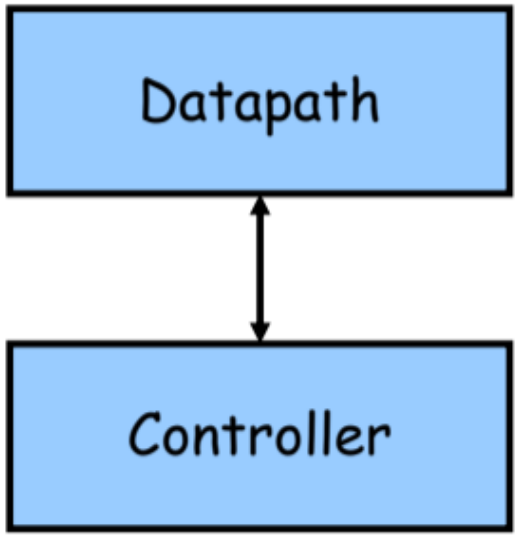
Lehung Nguyen

Reference

- [1] ECE499_Project.pdf
- [2] ProjectPresentation24.pdf
- [3] Basys 3 reference manual:
  - https://digilent.com/reference/programmable-logic/basys-3/reference-manual?srsltid=AfmBOoryXWaraevJBYB0y59GFjss0ZI9fc3AvaOE-CWx_SDR_5BmwsUy

Processor Architecture

- 1)Datapath
  - Includes components, alu, register file, memory, …
- 2)Controller
  - Controls flow of instruction and data in datapath



A. Datapath: the muscle

- The datapath contains all the functional units that hold or process data. It *does the heavy lifting but has no decision-making* power.
- Register File: (workbench): small, ultra-fast memory bank *(r0–r7)* inside the CPU. It holds the variables the CPU is currently working on. You cannot do math directly on data in RAM; *you must move it here first*.
- *ALU (Arithmetic Logic Unit):* Calculator. digital circuits that performs math (Add, Sub) and logic (AND, OR). It takes two numbers, crushes them together, and spits out a result
- Memory (Data & Instruction)
  - *Instruction Memory:* Holds the 'To-Do List' (your program code).
  - *Data Memory:* Holds the 'Inventory' (variables, arrays, large data) that doesn't fit on the workbench.

Instruction Memory

We need a container to hold instructions

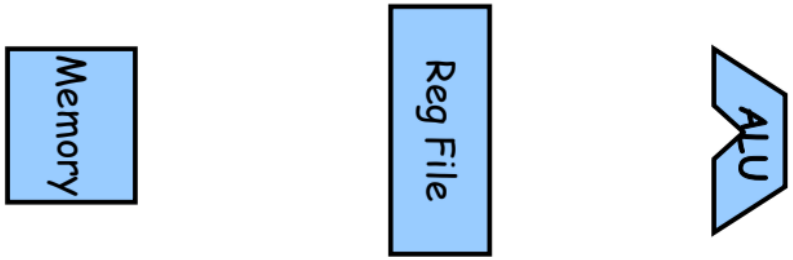
Please see slide 4 and subsequent discussing memory specifications

Register File

A place for r0…r7

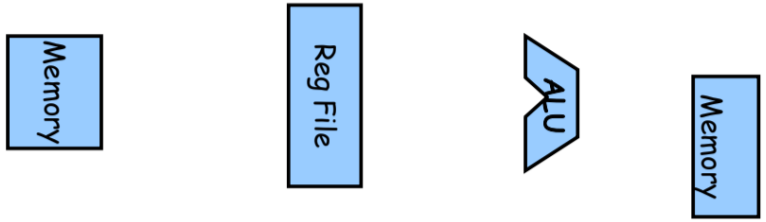
Alu

A unit for arithmetic calculations



Data Memory

A unit that holds data

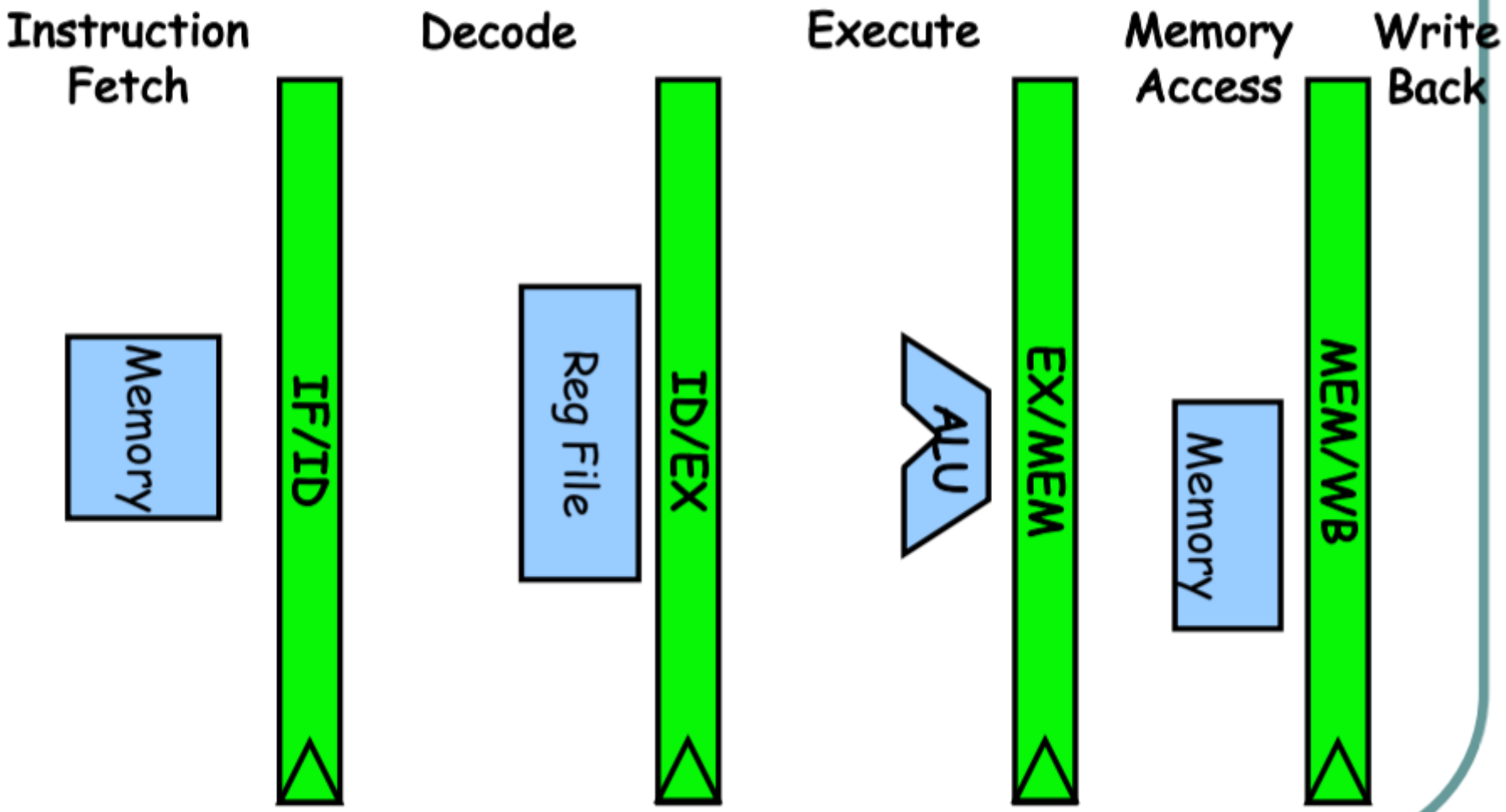


Controller

- Controller (Brain) manages the flow of information
- Controls Flow of Instruction: It looks at the Program Counter (PC) to decide which instruction to fetch next
  - Normally PC+1, or jumping to a new spot if it's a Branch instruction).
- Data in Datapath: manages the traffic lights. If the instruction is "ADD", the Controller flips the switches to connect the Registers to the ALU. If the instruction is "STORE", it flips switches to connect Registers to the RAM.

Pipeline Architecture

To break critical path



Pipelining

- Standard technique used to make modern processors fast. It divides the execution of an instruction into five distinct steps or "stages".
- The Critical Path (Single-Cycle CPU): If a processor that *doesn't* pipeline, To execute one instruction (like *ADD),* signals must travel through the PC → ROM → Reg File → ALU → RAM → Write Back all in one single tick of the clock. This is a very long path (e.g., 10 nanoseconds). Because the clock cycle must be long enough for the slowest instruction to finish, your CPU would be forced to run slowly (e.g., 100 MHz)

Pipelining

- *Pipelining (Breaking the Path):* Pipelining inserts "registers" (temporary storage) between each stage. Now, the signal only needs to travel through *one stage* (e.g., just the ALU) before the clock ticks.
  - Instead of one big step taking 10ns, you have five small steps taking 2ns each.
  - Result**:** You can run the clock 5 times faster (say 500 MHz) because the "critical path" is now only the length of the slowest single stage, not the whole chain.
- E.g.: doing laundry.
  - *No Pipelined:* You put clothes in the Washer, wait, then Dryer, wait, then Fold. You don't start Load 2 until Load 1 is completely folded.
  - Pipelined: As soon as Load 1 moves to the Dryer, you put Load 2 into the Washer. You are "breaking the path" so multiple loads are being processed simultaneously.

Explanation of the 5 Stages

- classic 5-stage *RISC* pipeline
- 1. Instruction Fetch (IF)
  - Get the instruction. CPU looks at the *Program Counter (PC),* sends that address to the Instruction Memory (ROM/RAM), and retrieves the 16-bit binary code
  - Hardware: PC, Instruction Memory.
- 2. Instruction Decode (ID)
  - Understand the instruction and get ingredients.
  - The Controller reads the Op-code to decide what to do. Simultaneously, it reads the *Register File* to pull out the data values (operands) stored in source registers (e.g., value 5 inside r2)
  - Hardware**:** Controller, Register File

Explanation of the 5 Stages

- classic 5-stage *RISC* pipeline
- 3. Execute (EX) Do the math The *ALU* takes the values found in the Decode stage and performs the operation (Add, Subtract, NAND).
  - *For Memory instructions:* It calculates the target memory address e.g r2 + offset
  - HW: ALU
- 4. Memory Access (MEM)
  - Interact with RAM (if needed).
  - If the instruction is LOAD/STORE the CPU reads or writes data to the Data RAM.
  - If the instruction is just math (like ADD) this stage does nothing—the result simply passes through to the next stage
  - HW: Hardware**:** Data RAM.

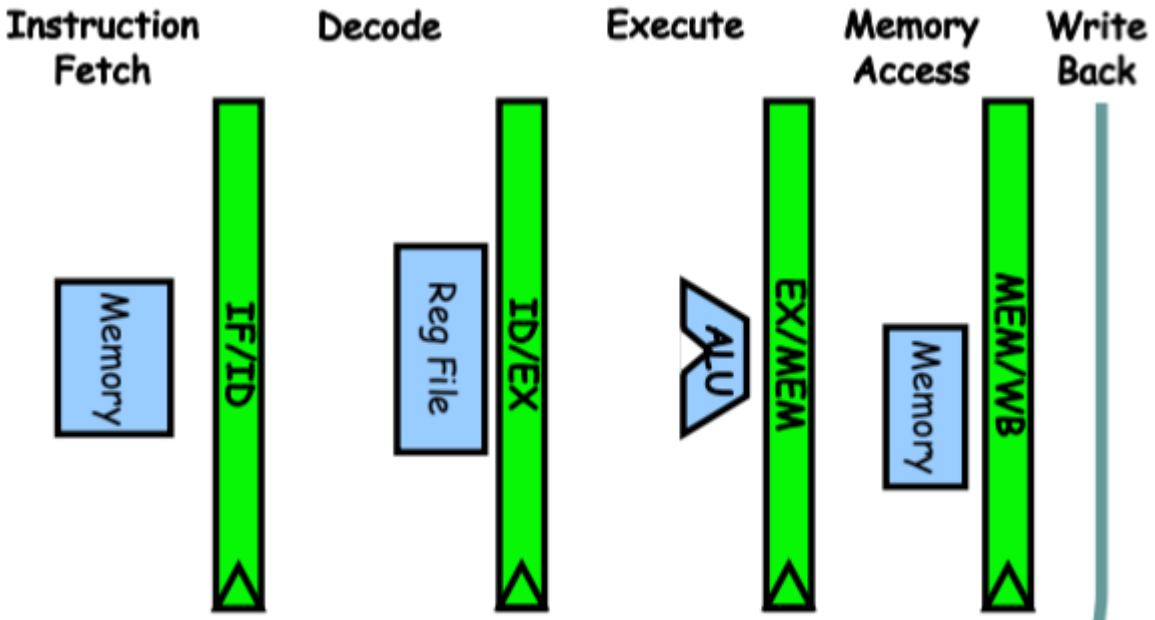
Explanation of the 5 Stages

- 5. Write Back (WB): Save the result. The final result (calculated by the ALU or read from Memory) is written back into the destination register(ra) in the Register File so it is ready for future instructions
- HW: Register File (Write Port).

5-Stages Datapath

Up to now, design of main components

Complete the datapath for every instruction gradually



5-state data path

- stop thinking about individual parts (like "just the calculator") and start thinking about the *entire system* (how the parts talk to each other).
- 1. "Up to now, design of main components' : Before you try to wire the whole CPU together, you must have completely finished and tested the individual "leaf" modules.
  - You should not move past this slide until you have working VHDL modules for
    - ALU**:** Can it add 5+5 and output 10?
    - Register: Can write r1 and read it back?
    - PC: Does it increment correctly?
  - Why? If you try to build the whole pipeline with a broken ALU, you will never know if the bug is in your wiring or in the math unit itself.

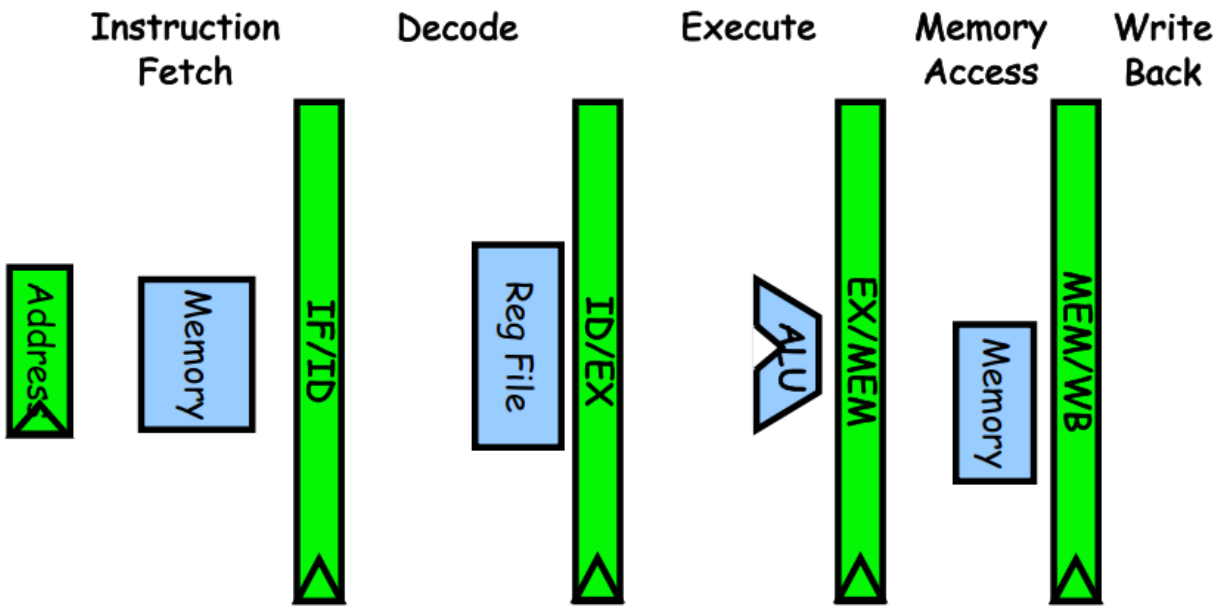
5-state data path

- IMPORTANT: do not try to support all instructions at once.
- Advice: ‘Gradual’ strategy: **Complete the datapath for every instruction gradually**
  - Instead of writing a massive Controller that handles ADD, SUB, LOAD, STORE, and BRANCH all at the same time, you should build the CPU to support *one instruction only*. Once that works, add the second one.
- Recommended Step-by-Step Flow
  - *Iteration 1 (The Skeleton)*: Wire the Datapath so it can *only* do an *ADD* instruction. As slide 27 in [1]
  - *Iteration 2 (The Math Class):* Once ADD works, enable the other A-Format math instructions (SUB, NAND). This is easy because they use the exact same path, just different ALU modes.
  - *Iteration 3 (The Logic Class):* Add support for *Branching*. Now you have to wire the PC to accept jumps.
  - *Iteration 4 (The Memory Class):* Finally, add the *LOAD/STORE* logic. This is the hardest part because you have to pause execution to talk to RAM.
- Why follows this ‘gradual’ strategy: if your CPU breaks in Iteration 3, you know exactly where the problem is (the branching logic), because you already proved the math logic worked in Iteration 1.

PC [1] slide 28

PC

A component that holds address of Inst. Memory (PC)



Instruction fetch stage

- This slide represents the very first step of the pipeline: the *Instruction Fetch (IF)* stage: how the processor physically retrieves a command from storage so it can begin processing it.
  - "A component that holds address of Inst. Memory (PC)" mean: defines the most important register in your entire processor: *Program Counter (PC)*.
  - *Inst. Memory: Instruction Memory*. In your project, this is the ROM (for BIOS) or Port B of the RAM (for User Code). It is the "book" containing all the instructions.
  - *"Holds address":* The PC does not store the instruction itself (it doesn't store "ADD"). Instead, it stores the *location* (the page number) where that instruction lives.
  - Instruction Memory as a giant library of books (instructions). The *PC* is the slip of paper in your hand with the "Call Number" (Address) written on it. You cannot get the book without holding that slip of paper.

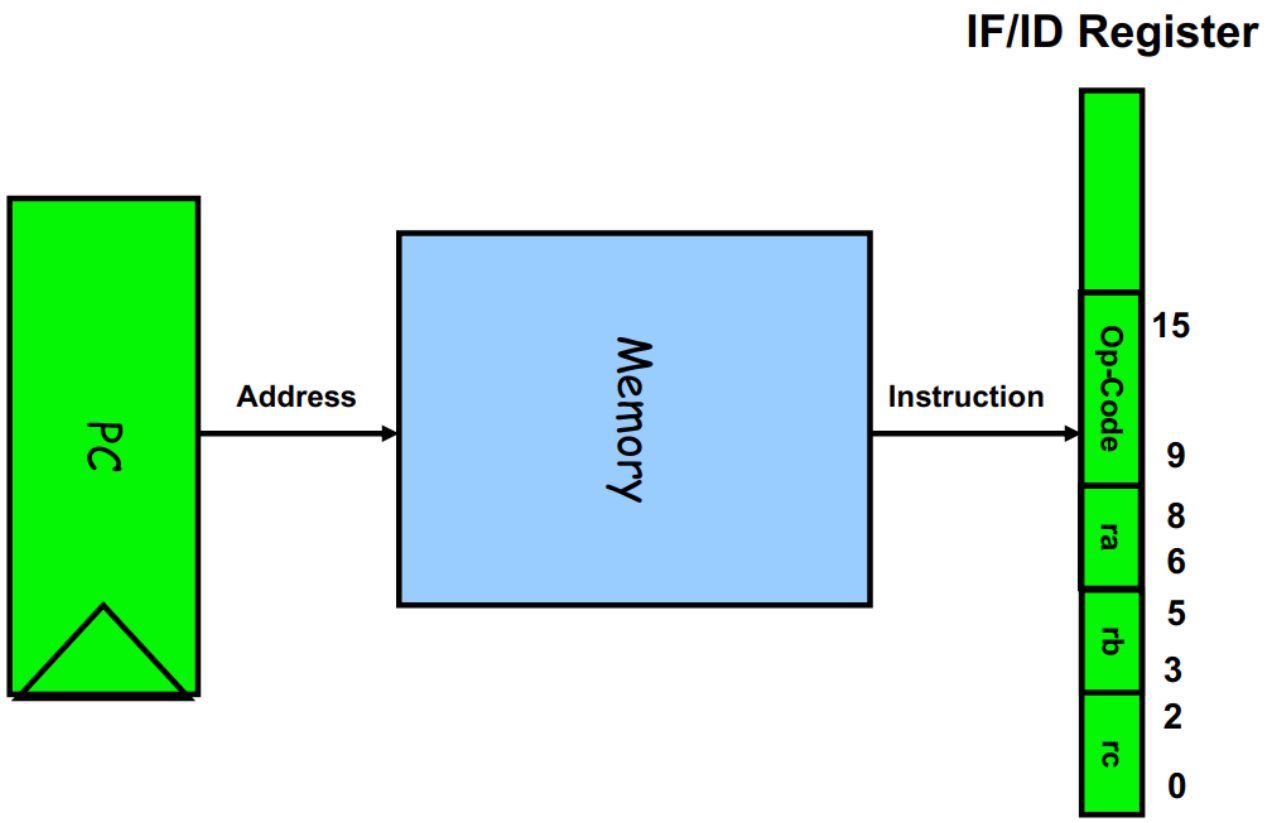
Instruction fetch stage

This slide shows data flow for fetching a single instruction:

- *1. The Source (PC/Address):* The green bar on the far left labeled "Address" represents the *Program Counter*. (e.g. it holds the number 0x0400)
- 2. *The Lookup (Memory):* This address (0x0400) travels along a wire into the blue box labeled "*Memory*". The memory hardware looks at slot 0x0400 and finds the binary code stored there (e.g., 1001… for a LOAD instruction)
- 3. *The Handoff (IF/ID):* The memory outputs that binary code. The code is captured and saved into the long green vertical bar labeled *IF/ID: Instruction Fetch / Instruction Decode*.
  - It is a "pipeline register" that acts as a temporary holding area, holds instruction steady so the *next* stage (Decode) can look at it during the next clock cycle.
- In code for this stage, simply connect the output of your *PC module* to the address input of your *Memory module*, and connect the data output of the Memory to the input of the *IF/ID register*.

Slide 29 [1] Fetch

Fetch



Slide 29 [1] Fetch

- Illustrates the flow of a specific instruction (an ADD command) as it enters the processor.
  - 1. PC (Program Counter): The PC holds the memory address where the current instruction lives.
  - 2. Memory Access: The PC sends this address to the Instruction Memory.
  - 3. Instruction Retrieval: The memory outputs the 16-bit binary code stored at that address
  - 4. Latching into IF/ID: *critical*: The 16-bit instruction does not go directly to the decoder. Instead, it is saved (latched) into the *IF/ID Register*.
    - *Why?* This saves the instruction so the Fetch stage is free to grab the *next* instruction (PC+1) in the very next clock cycle, while the Decode stage works on this one.

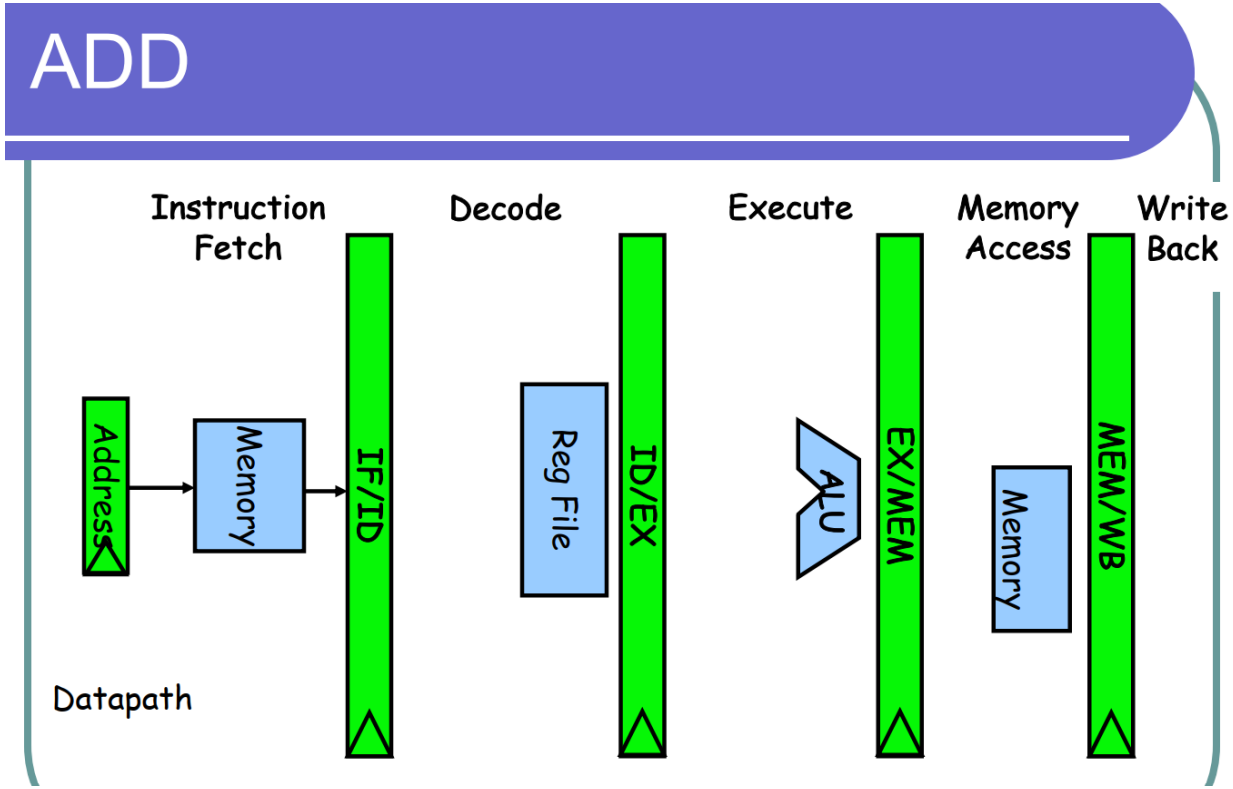
IF/ID register & normal registers (r0-r7)

- the IF/ID register is NOT the same as the normal registers (r0-r7)
- They serve completely different purposes and live in different parts of the hardware.

Feature	Normal Registers (r0-r7)	Pipeline Registers (IF/ID, ID/EX)
Visibility	*Visible to You*. You write assembly code that explicitly names them (e.g., ADD r1, r2).	*Invisible*. You never write code that touches them. They are hidden wiring inside the CPU.
Purpose	*Data Storage*. They act as the "workbench" for holding variables (numbers) your program is calculating.	*Time Travel*. They act as "waiting rooms" that hold an instruction and its data steady as it moves between stages.
Location	Grouped together in the *Register File* module. +1	Placed physically *between* the stages (e.g., between Memory and Decode).
Size	Each is 16 bits wide.	They are extremely wide (often 50+ bits) because they must hold the *entire* state of the instruction (PC, Instruction bits, Control signals, Data).

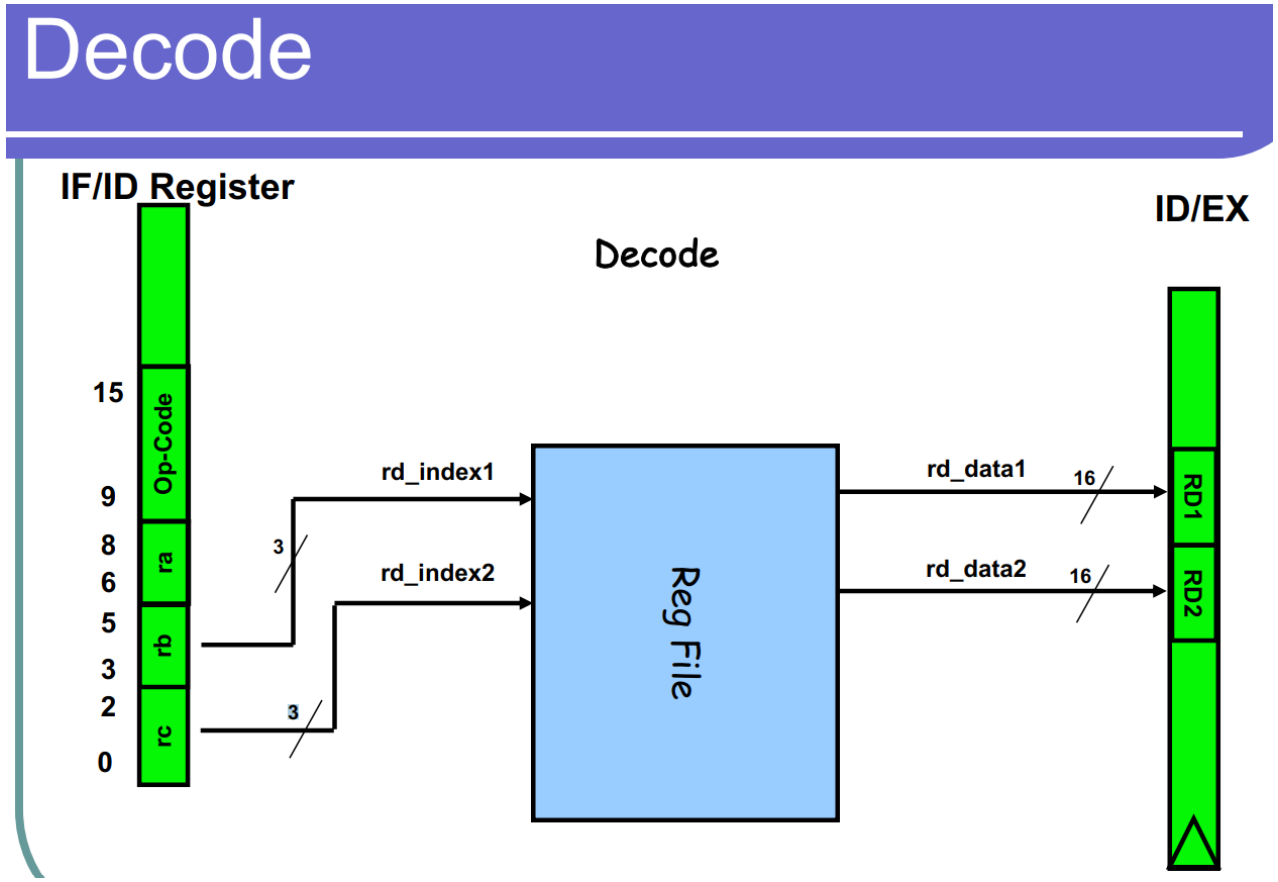
Slide 30 [1]

- *Decode Stage* second step of the pipeline
  - The instruction (specifically an ADD command) has successfully landed in the *IF/ID Register* (the vertical green bar on the left)
  - system is now ready to "Decode" this instruction. The blue box labeled *Reg File* : next step is to look inside the instruction, find the register addresses (ra, rb, rc), and fetch their values from the register file.

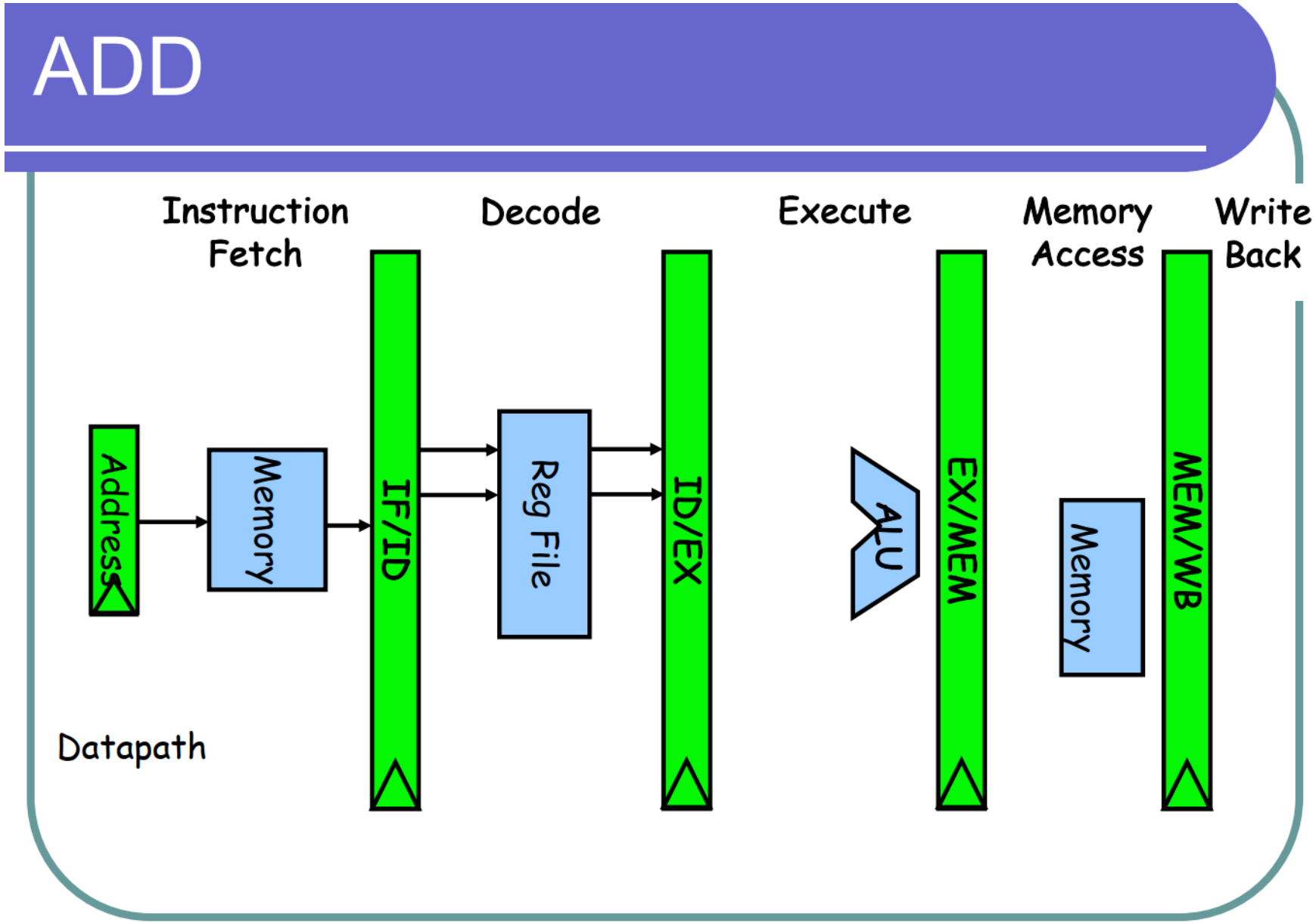


Slide 31 [1] Decode

- *IF/ID Register:* holds the instruction fetched in the previous cycle
  - "splits" 16-bit instruction into chunks based on the *A-Format*
- *rd_index1, 2* (Read Index 1,2): ***3 bit register index***
- *Register File:* translates the binary 010 and 001 into the meaning: *"I need to grab the data from Register 2 and Register 1."*
- *rd_data1, 2:* full **16-bit** "true value" extracted from the register
- *Decoding:* It translates the binary 0000001 into the meaning: *"I need to configure the ALU to perform ADDITION."* (This happens in the Controller, which works parallel to this slide).

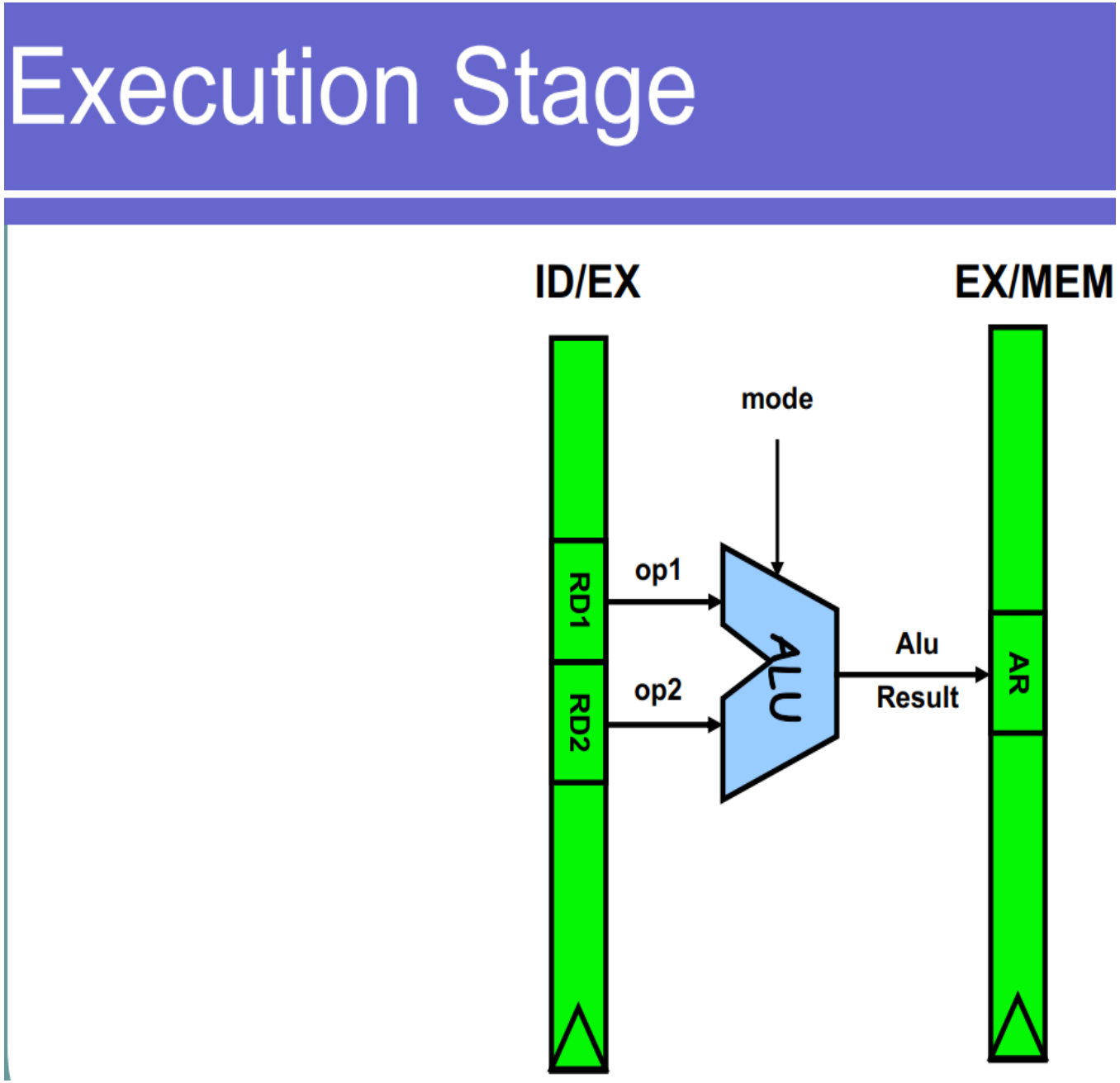


Slide 32/33 [1] Execute Stage (ADD Instruction)



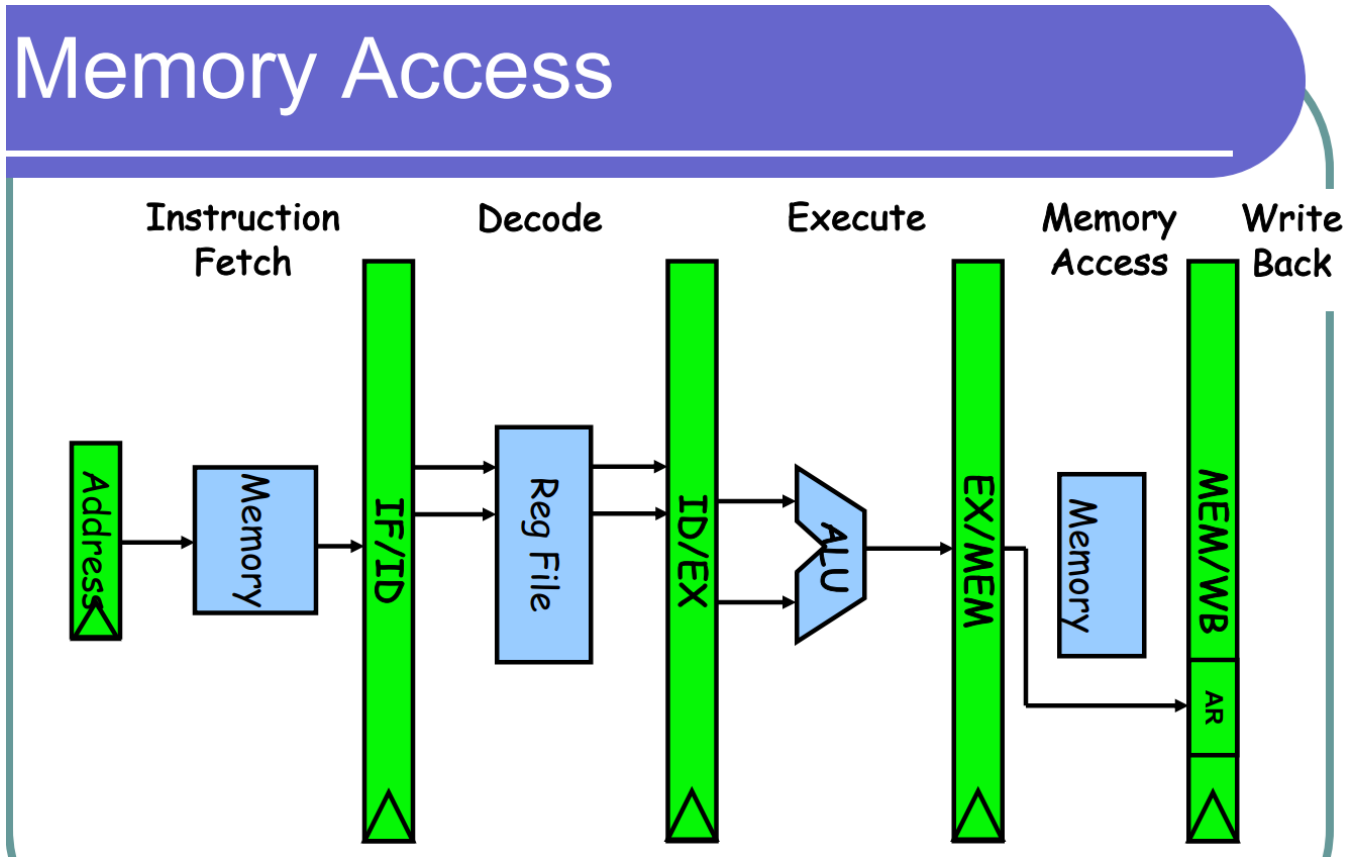
Slide 32/33 [1] Execute Stage (ADD Instruction)

- 1. ID/EX Register
  - holds the two data values we fetched in the previous cycle (labeled RD1 and RD2)
- 2. ALU (Arithmetic Logic Unit)
  - Controller sends a signal to the ALU saying "Mode: ADD”
- 3. EX/MEM Register (Execute / Memory)
  - Save result of the math into pipeline register
  - Pass to *next stage (Memory/Write Back)* in the next clock cycle.
  - ADD: ***no need to interact read/write*** from/to data RAM (like L-Format Load/Store)



Slide 35 [1] Memory access

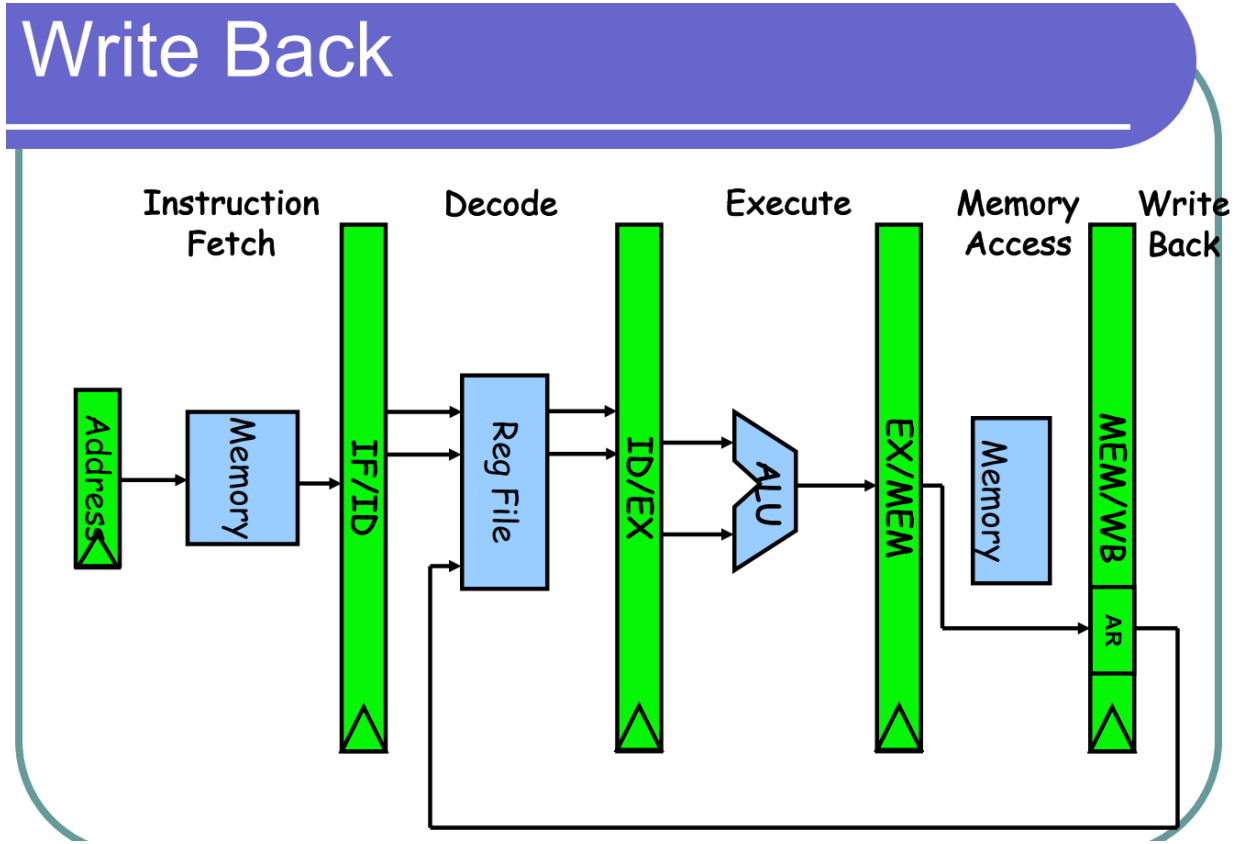
- In this stage, the processor decides whether it needs to interact with the main Data Memory (RAM) or just pass the calculated result along.
- The ADD instruction is purely mathematical. It does not need to save data to RAM or read data from RAM.
  - The ALU result (e.g., 8) simply "passes through" this stage without activating the memory.
  - It travels from the EX/MEM register, bypasses the blue "Memory" box, and lands in the MEM/WB register .



Slide 36 [1] Write back

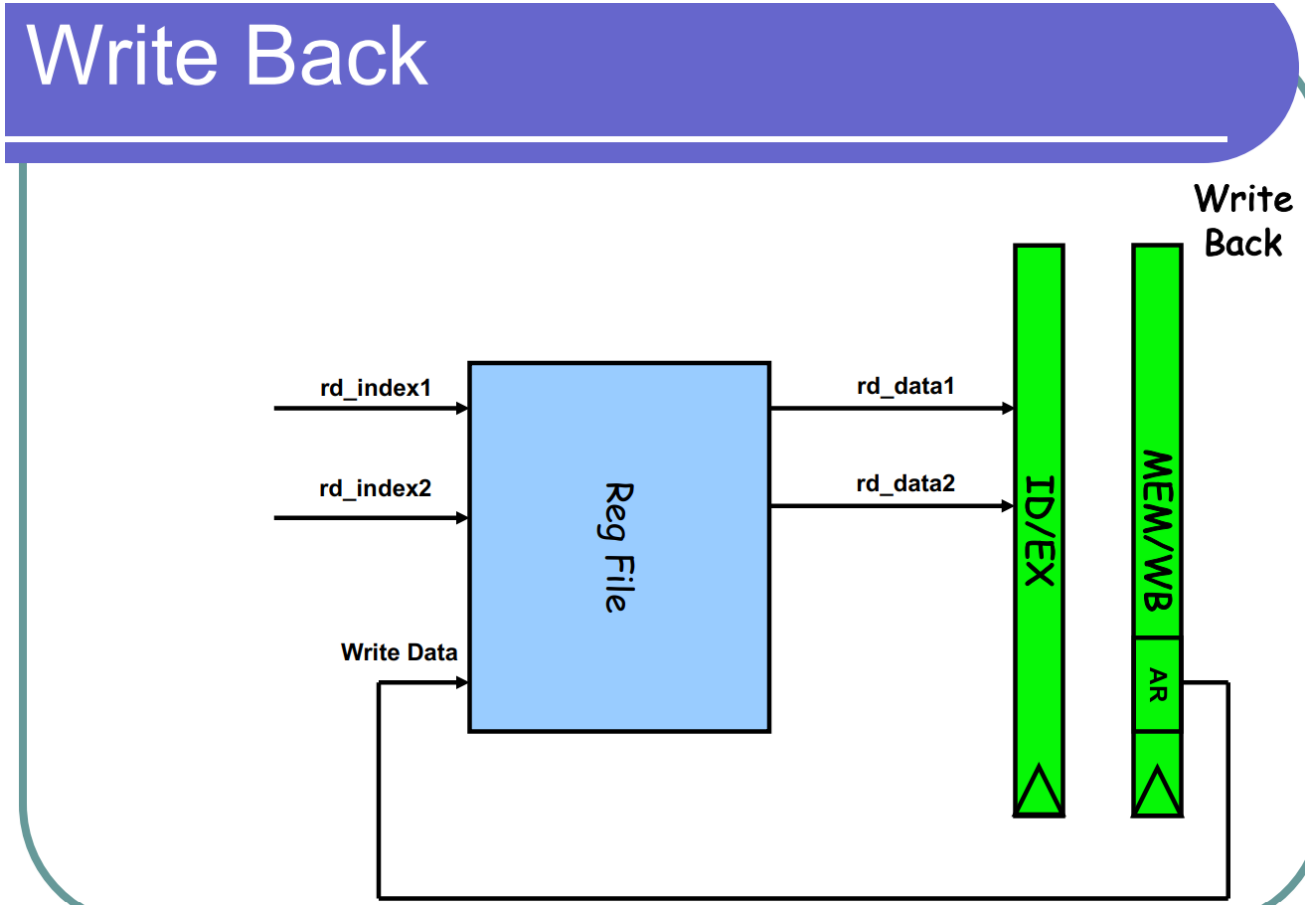
The Setup (The Data Arrives)

- Input: the *MEM/WB* (Memory/Write Back) pipeline register
- *Payload:* Inside this register is the value *AR* (ALU Result). In our ADD example, this holds the calculated number (e.g., 8)
- The processor needs to take this 8 and put it safely into the destination register (e.g., r3).



Slide 37 [1] Write back

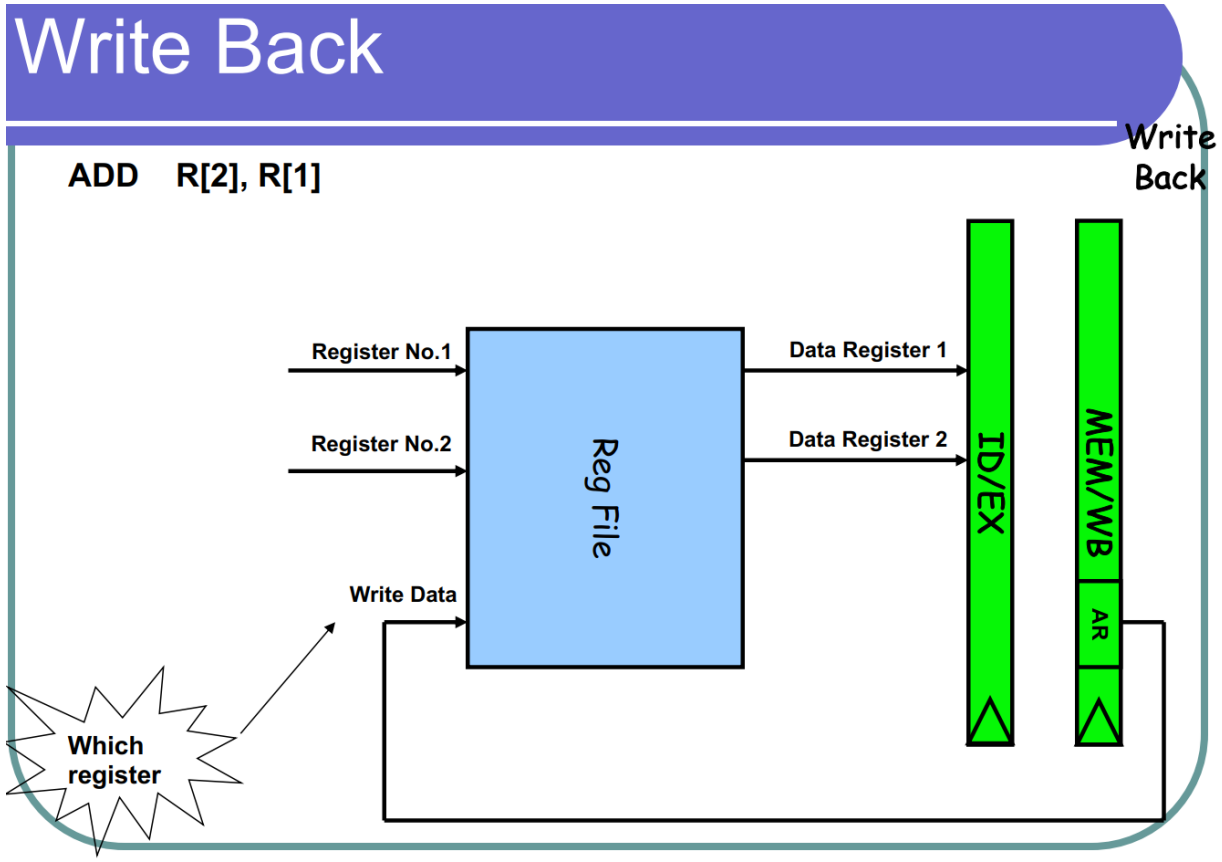
- A wire connects the *AR* slot in the MEM/WB register all the way back to the *Write Data* port of the *Register File*
- Result ‘8’ travels along this wire, waiting at the "gate" of the *Register File* to be written.



Slide 38 [1]

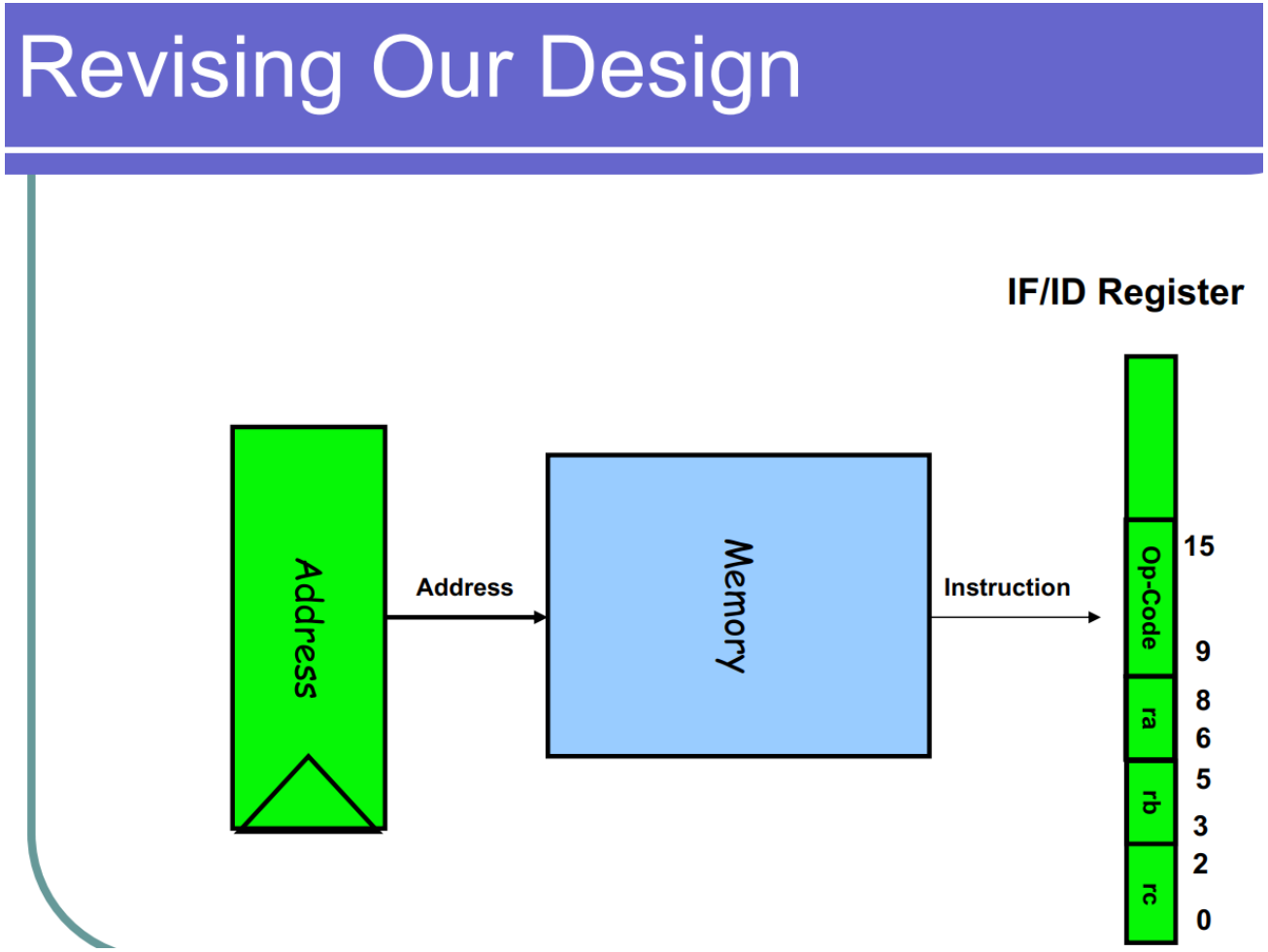
Problem: Which Register?

- the *Data* (the number 8) ready at the Write Data port.
- But the Register File also needs an *Address* at the Write Index port to know *where* to put that data.
- The instruction *ADD r3, r2, r1* was decoded 3 clock cycles ago.
  - "r3" used in the *Decode* stage but *not* passed forward in our current diagrams.
- Solution: *must modify your pipeline registers (ID/EX, EX/MEM, MEM/WB)* to carry the *Destination Address (ra)* along with the data.



Slide 39 [1] Revising Our Design

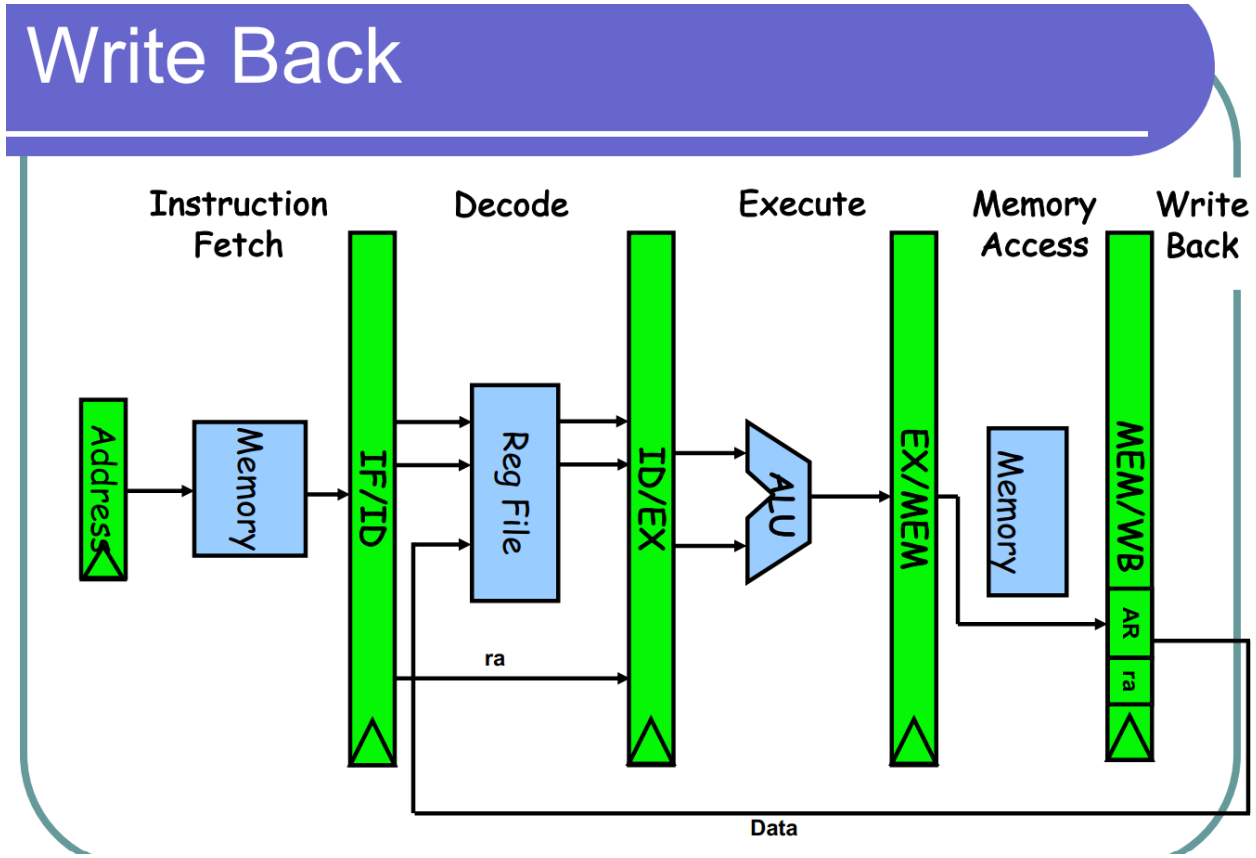
- By the time *an instruction reaches the end of the pipeline (Write Back)*, the processor has "forgotten" which register was supposed to receive the result.



Slide 40 [1]

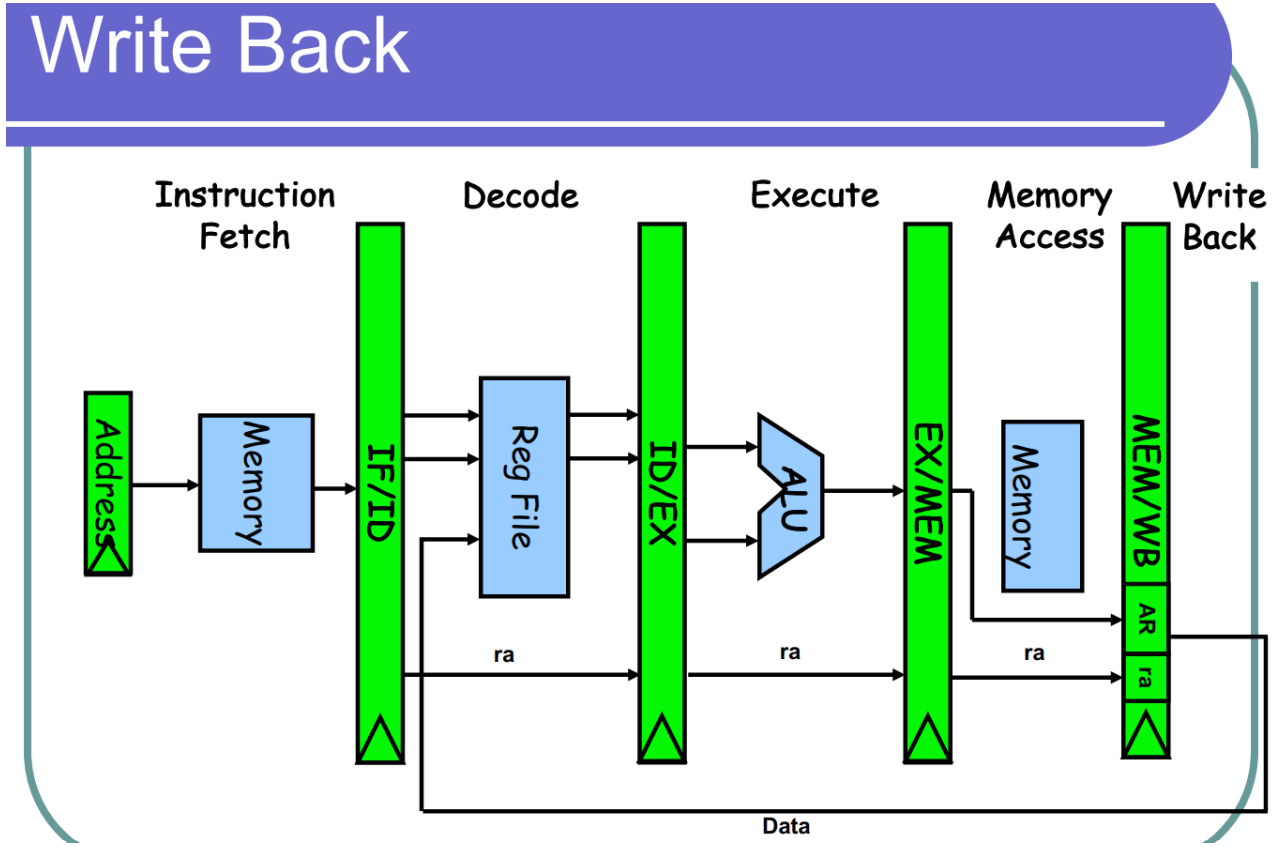
Instruction ADD r3, r2, r1.

- *Time T=2 (Decode):* The processor knows the destination is r3 (bits 8-6 of the instruction).
- *Time T=5 (Write Back):* The instruction is finishing. The result (e.g., 8) is ready to be saved.
- *Issue:* At Time T=5, the Decode stage is busy decoding *Instruction #4*. The "instruction wires" are *currently holding the bits for Instruction #4*, not our original ADD
- Mistake: connect the current instruction wires to the Write Back stage, save the result of the *first* instruction into the destination register of the *fourth* instruction. corrupt



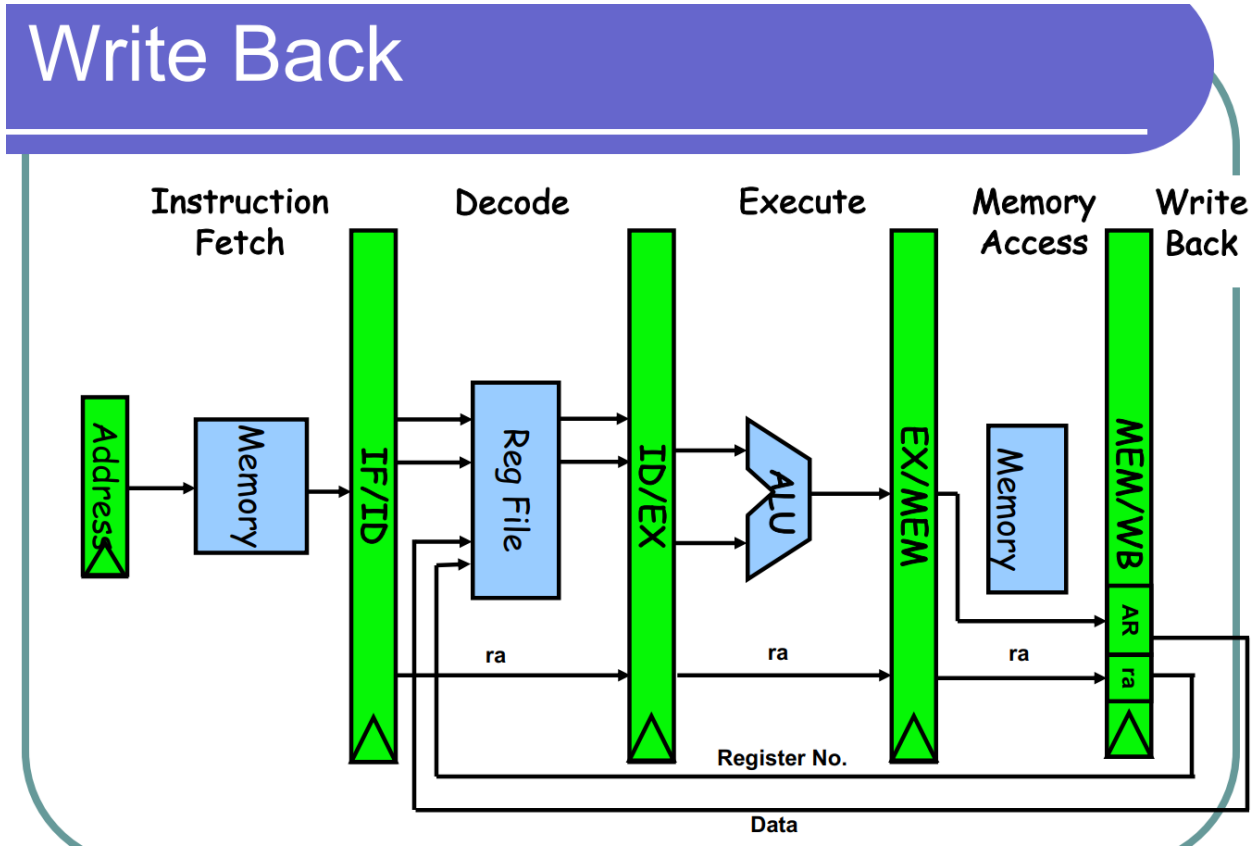
Slide 40,41,42 [1]

- Solution: carry the *destination address* (r3) *through the pipeline alongside the data,* just like putting a name tag on a suitcase so it doesn't get lost.
- Add 3 extra bits to every register buffer (ID/EX, EX/MEM, MEM/WB) specifically to hold the *Destination Index (ra)*.



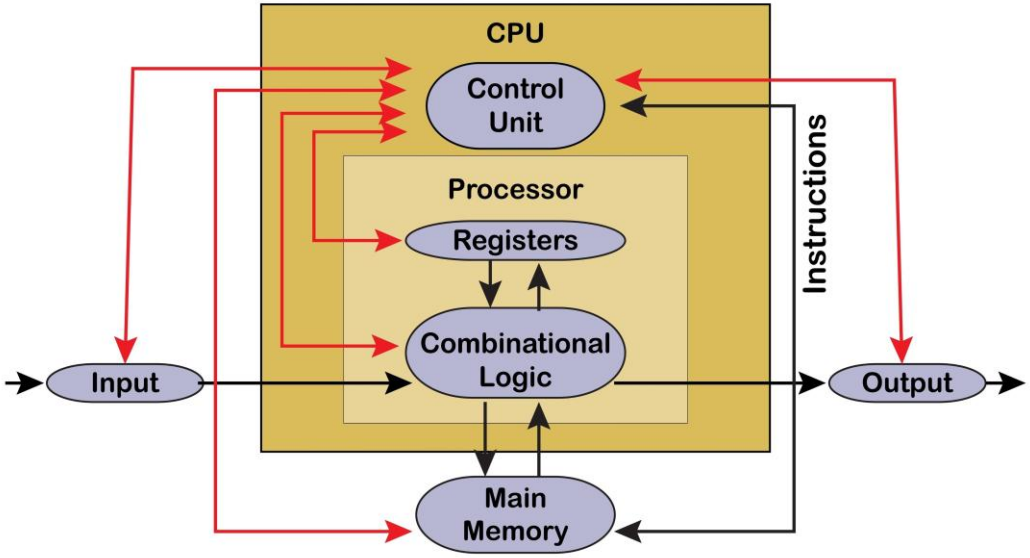
Slide 43 [1]

- Because the address r3 traveled *with* the data through every stage, they *arrive at the Register File at the exact same moment*. The Register File now knows exactly where to write the data.
- When you define your pipeline register modules, you must *include a 3-bit input/output specifically for reg_dest*. You *must pass this signal through every stage, even though you don't "use" it until the very last step*.



Slide 45 [1] Controller

- So far, we have been building the *"body"* of the computer (the *Datapath*: registers, ALU, memory). Now, we are adding the *"brain*.“
- Controller: a block of digital logic (usually a large decoder or a look-up table) that tells the *Datapath what to do*.
  - Input: reads the *Op-code*. sees 0000001 and knows "This is an ADD instruction.“
  - Output: generates *Control Signals*. These are single-bit wires (1 = On, 0 = Off) that flip switches throughout the CPU.

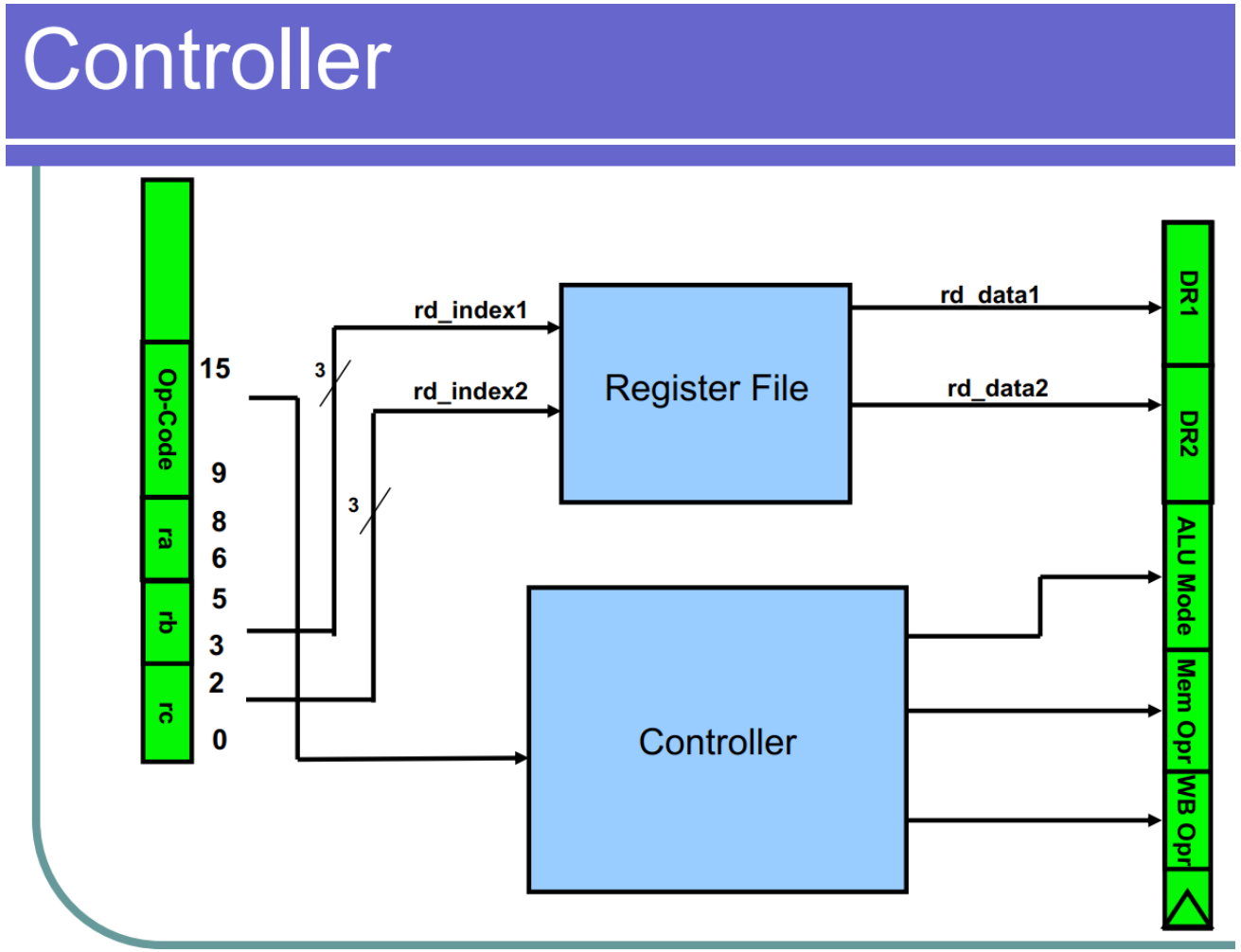


‘Distributed’, ‘hierarchical’, ‘centralized’ Controller

- 1. Centralized Control: one single, large Controller block (usually in the Decode stage). It looks at the instruction and issues *all* control signals for the entire pipeline at once. ***(used in standard 5-stage pipeline you are building (MIPS-style))***
  - Pros: Easy to design initially; all logic is in one place.
  - Cons: The controller becomes huge and complex; hard to run at high speeds because of long wires going everywhere.
- 2. Distributed Control: Every part has its own small brain
  - Instead of one big controller, each stage (Fetch, Decode, Execute, Memory) has its own small local controller. The instruction code travels down the pipeline, and each stage looks at it and decides what to do locally.
  - Pros: Faster (shorter wires); modular (easy to change one stage without breaking the others).
  - Cons: Duplicates logic (every stage needs to decode part of the instruction).
- 3. Hierarchical Control: A big brain gives general orders; small brains handle details
  - A main controller breaks the instruction into "Micro-Operations" (high-level tasks). It sends these tasks to smaller controllers in the Datapath, which handle the nitty-gritty details (like exactly which transistors to turn on).
  - *Pros*: Very organized; handles complex instructions (like "Save all registers") efficiently.
  - *Cons*: Most complex to design.

Slide 45 [1] Controller

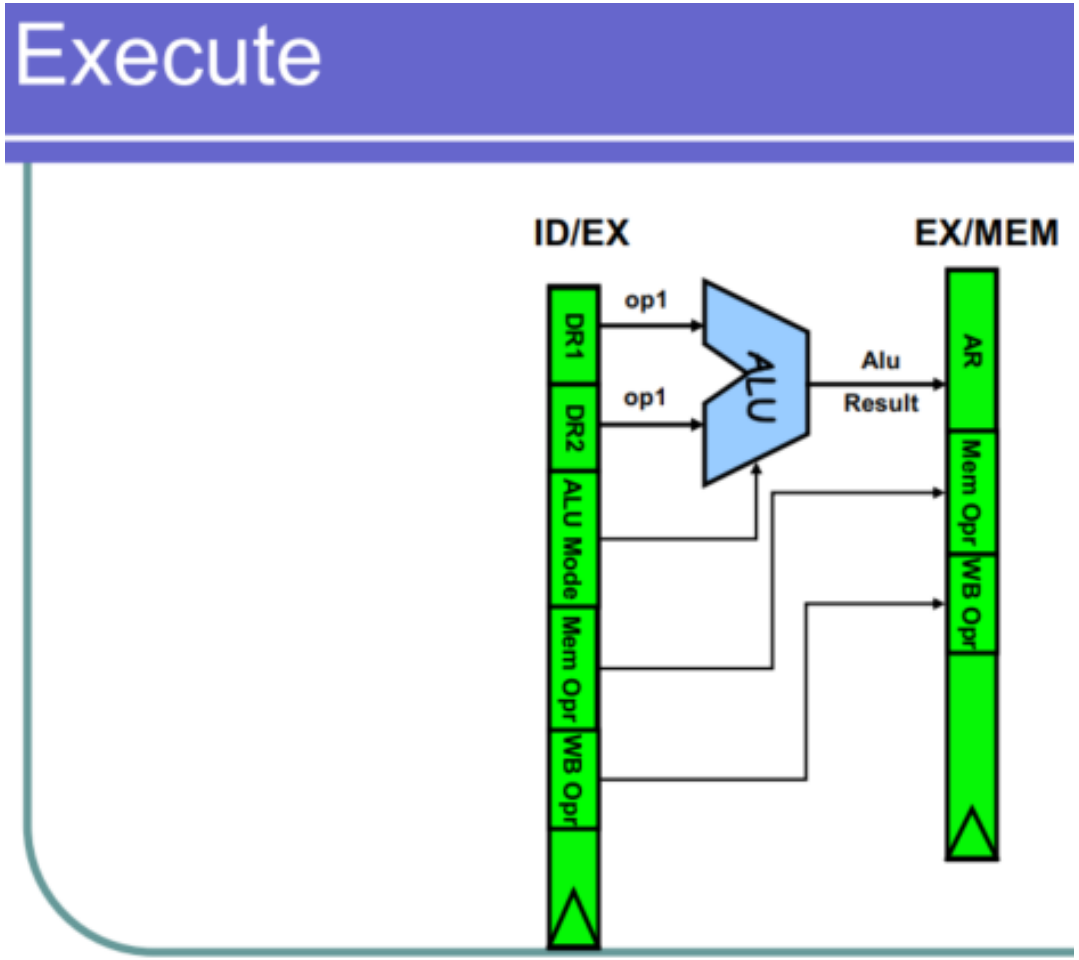
- The Controller makes *all* these decisions during the *Decode (ID)* stage.
  - The ALU needs its signal in the next stage (Execute).
  - The Memory needs its signal in the stage after that (Memory).
  - The Register File needs the "Write Enable" signal at the very end (Write Back).
- *Control Signals must travel down the pipeline registers* just like the data does
  - The ID/EX register saves the ALU signals.
  - The EX/MEM register saves the Memory signals.
  - The MEM/WB register saves the Write Back signals.



The controller controls all the stages of the pipeline.
It may be distributed, hierarchical or centralized

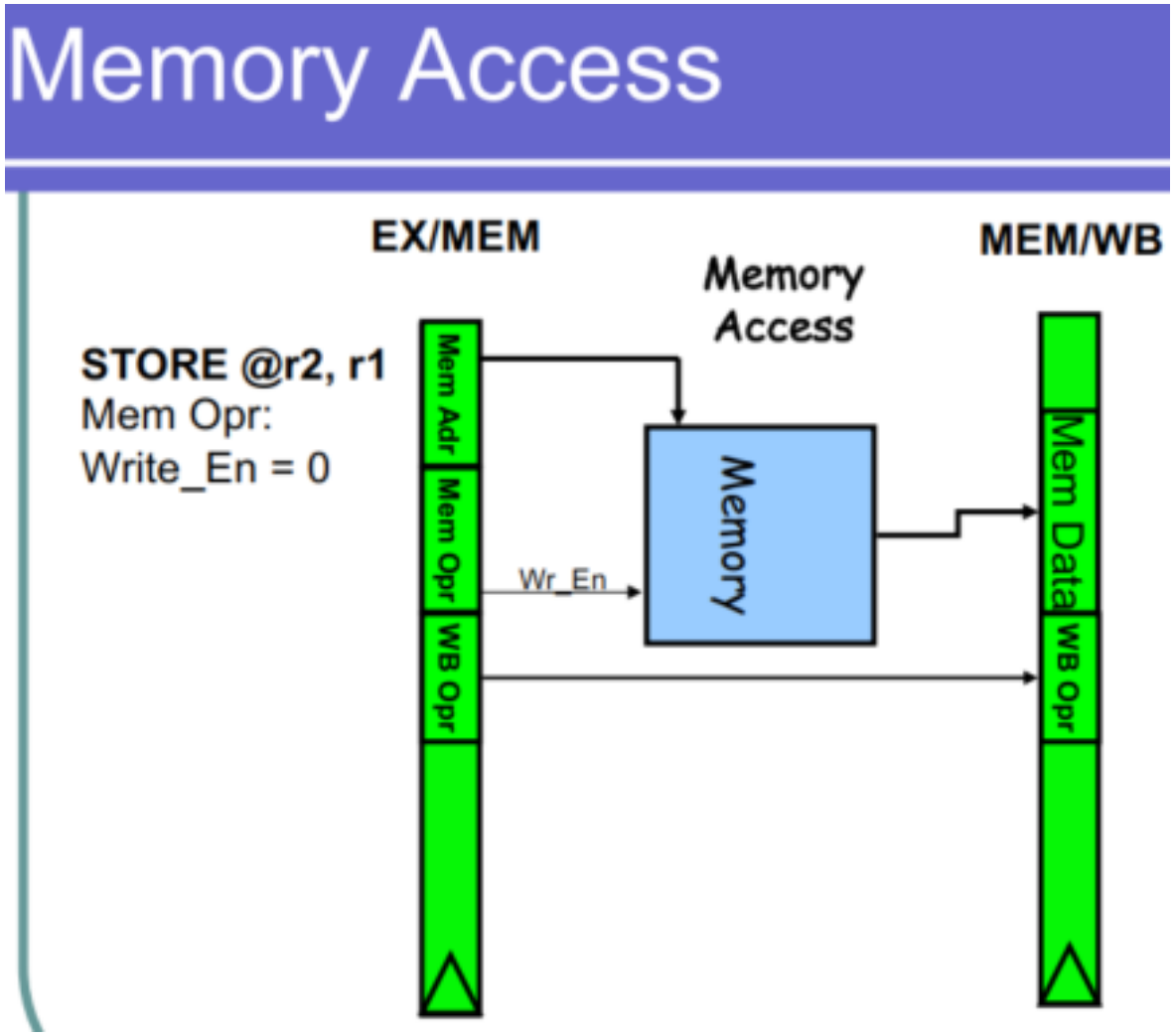
Slide 46 [1] Controller in Execute stage

- ALU mode (ALU Control Signals):
  - specific orders for the ALU (the calculator)
  - "Perform Addition," "Subtract," "Use Immediate value instead of Register.“
- Mem Opr (Memory Operations)
  - Orders for the *RAM* (Memory). The ALU doesn't care about them, so they are just "riding along”.
  - E.g.: Read from Memory (MemRead)," "Write to Memory (MemWrite).
- WB Opr (Write Back Operations)
  - Orders for the final *Register File* update. Should be carried until the very end



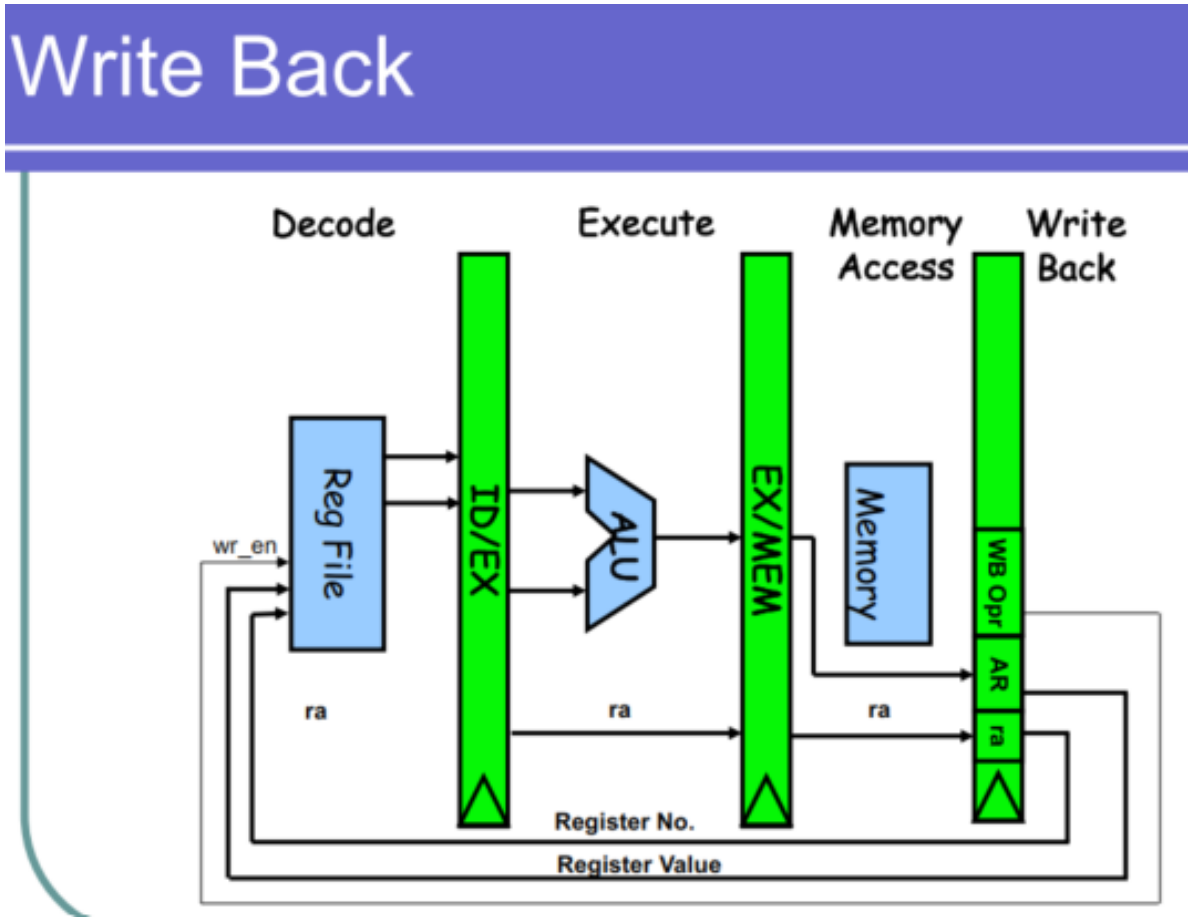
Slide 47 [1] Memory Access in Controller path

- *Memory Access (MEM):* for a STORE instruction.
- STORE @r2, r1: writes the content of register r1 into the memory address pointed to by r2
- Key Components:
  - *EX/MEM Pipeline Register*: Holds the calculated memory address (from the ALU) and the data to be stored
  - Write Enable (Wr En): A control signal from the controller that should be high to allow writing to RAM



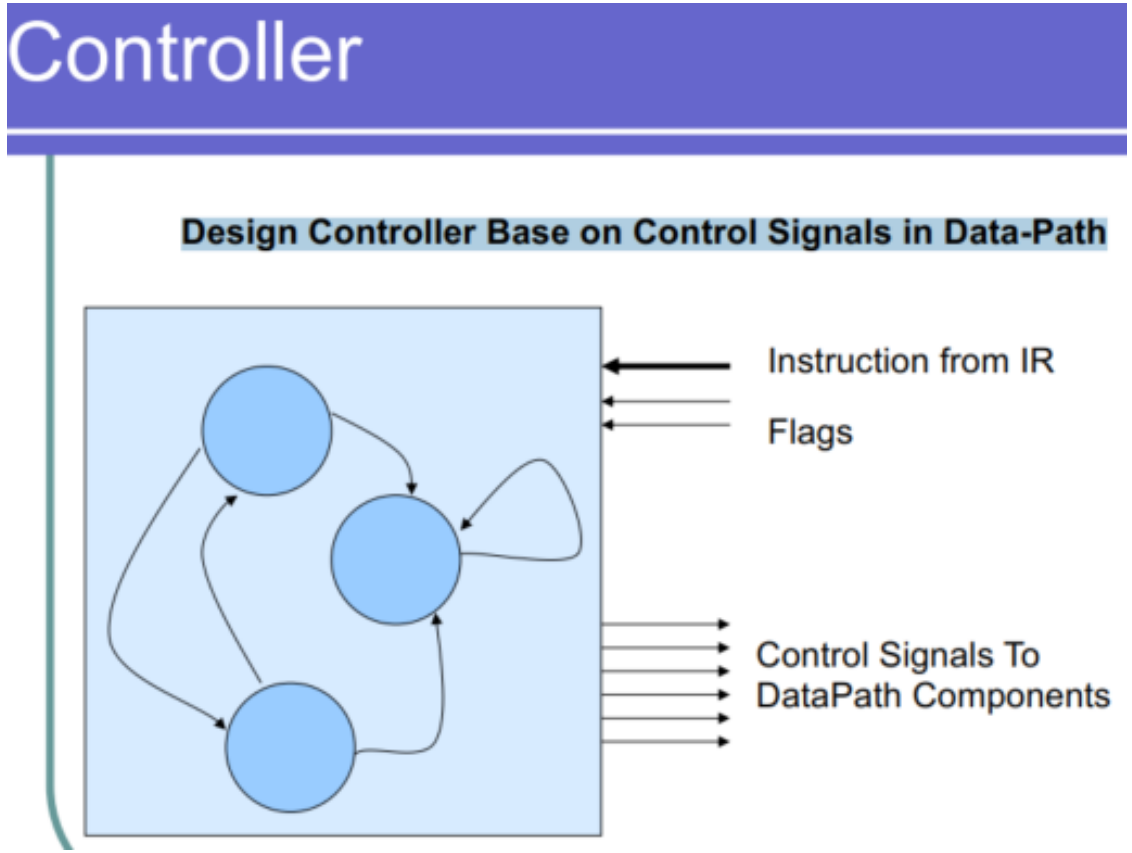
Slide 48 [1]

- WB Opr (Write Back Operation)
  - *control signal* generated by the Controller during the Decode stage
  - logic required to tell the Register File whether it should actually perform a write operation (wr_en)
- *AR (Address Result / ALU Result)*
  - *Result* coming out of the ALU (Arithmetic Logic Unit)
- ra (Register Address):
  - *index* (0 through 7) of the destination register
  - passed through every single pipeline stage. travels alongside the data so that when the result (AR) is finally ready to be written back, the CPU still knows which register it was supposed to go into.



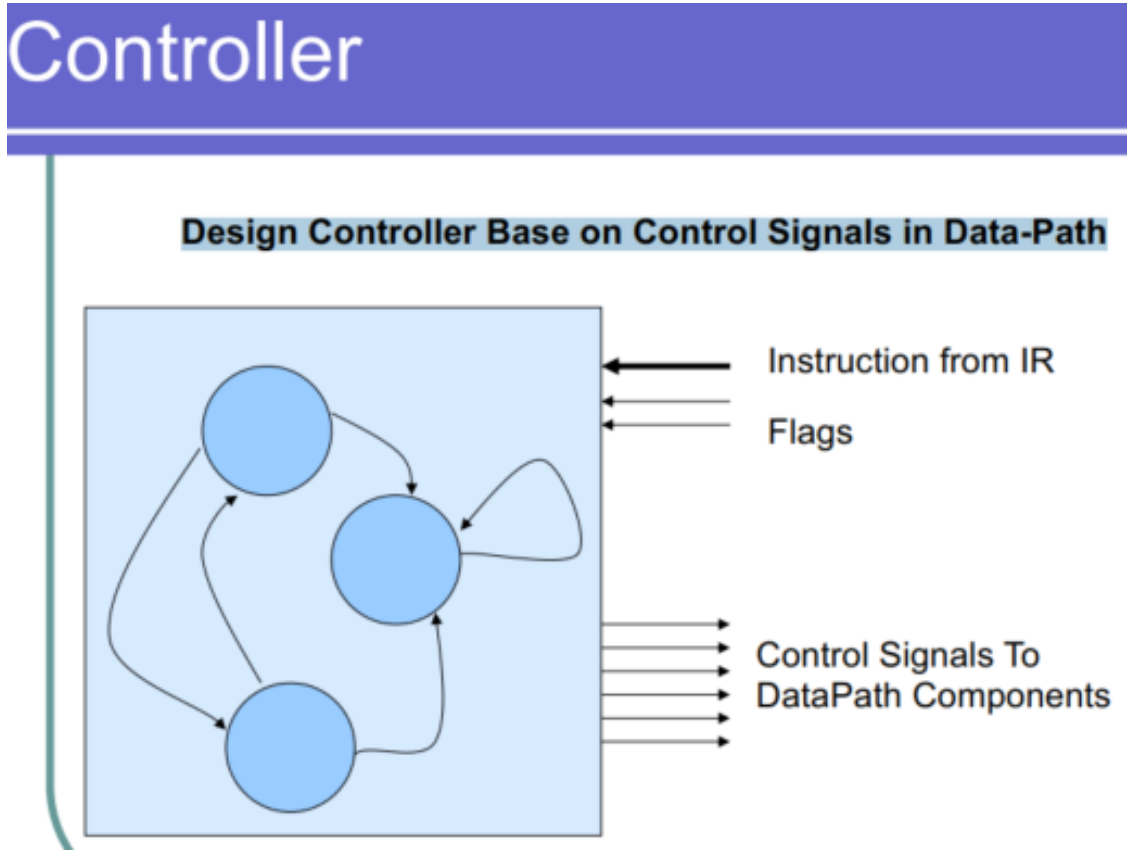
Slide 49 [1]

- Design Controller Based on Control Signals in Data-Path
  - The Datapath Dictates the Needs: Each of hardware modules (ALU, Register File, Memory) modules has "control pins“. E.g ‘Register File’ has a *wr_en* pin. ALU has a mode pin to choose between addition or subtraction.
  - “Design based on control signals" means you list every *single control pin you created in your datapath and then write the logic (usually a State Machine)* to drive those pins based on the Op-Code



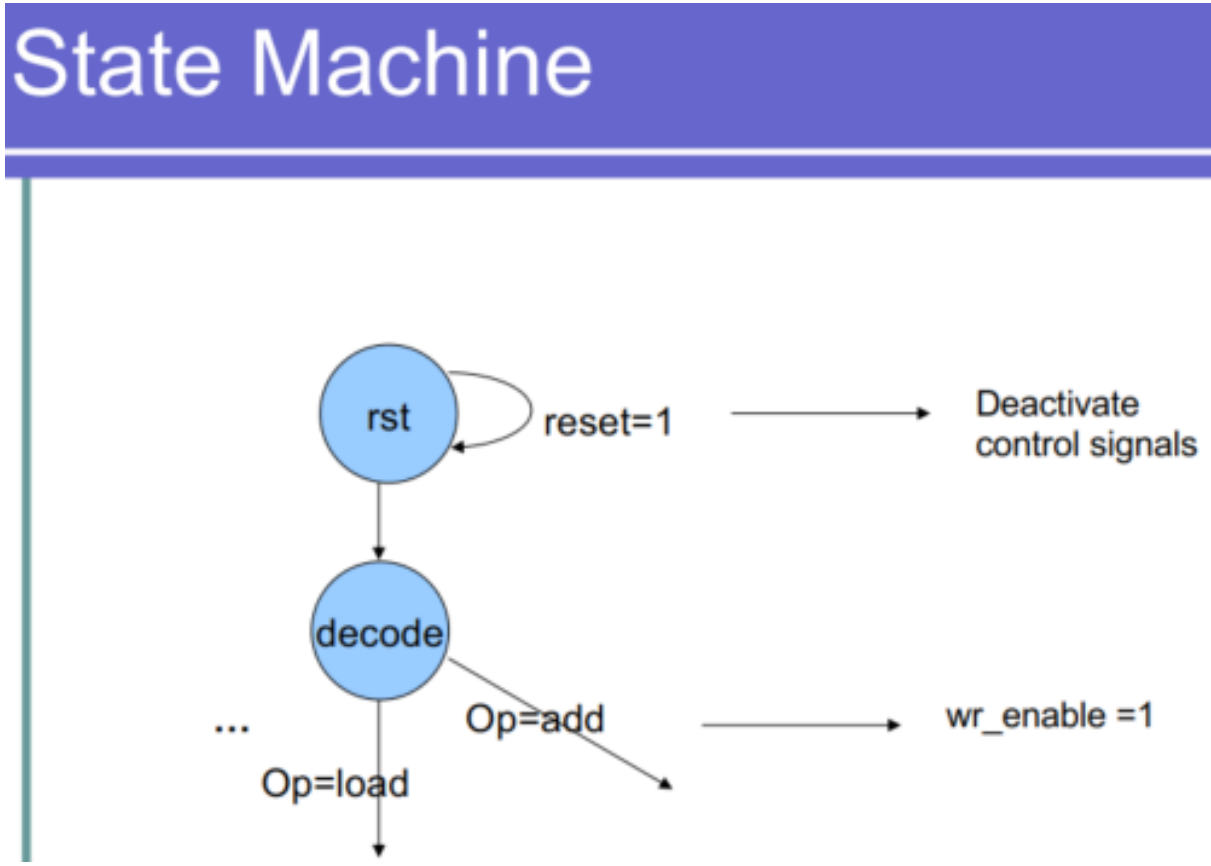
Slide 49 [1]

- *Input:*
  - *Instruction from IR (Instruction Register):* The Controller's primary input. It reads the Op-Code (bits 15:9) to know what the instruction is (e.g., LOAD, ADD, or BRR).
  - *Flags:* status bits like *Z (Zero) or N (Negative)*. The Controller needs these to make decisions for *Branch* instructions (Format B), such as BRR.Z (Branch if Zero).
- *Ouput:*
  - *Control Signals to Datapath Components:* physical wires that connect the Controller to the rest of the CPU to tell it exactly what to do



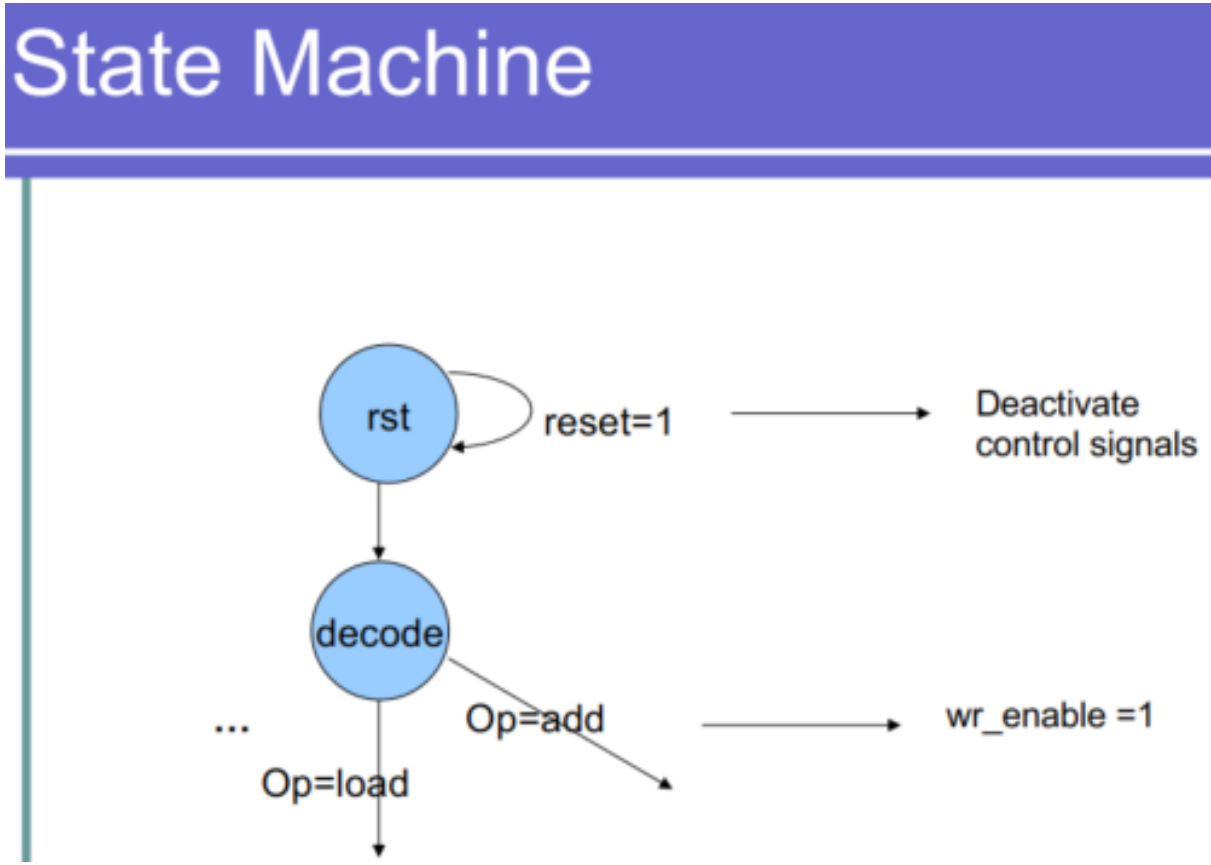
Slide 50 [1] State machine

- Design of the *Controller* using a *Finite State Machine (FSM):* "logic" that translates an instruction into the physical actions required to move data through the CPU stages
- *1. State: rst (Reset):* initial state of the controller when the system is powered on or a reset signal is received.
  - Deactivate control signals: forces all control signals (like memory write enable or register write enable) to a "safe" or inactive state (usually 0).
  - This prevents the CPU from accidentally writing garbage data to memory or registers during startup
  - *Trigger (reset=1):* As long as the physical reset button/signal is held high, the FSM stays in this *rst* circle



Slide 50 [1] State machine

- 2. State: decode: the controller looks at the *Op-code* bits of the current instruction to determine what the CPU needs to do next.
  - Arrows: different "branches" of logic depending on the instruction type.
  - *Op=add:* ADD instruction. wr_enable = 1. set the Register File's write-enable signal to 1 so the sum can be saved
  - *Op=load:* instruction is a LOAD, the FSM follows a different branch to manage memory reading instead of ALU math
  - *(...)*: many other paths not shown in the simplified diagram, such as those for *STORE, BRANCH, or SUB*



Slide 51 [1] State Machine Implementation

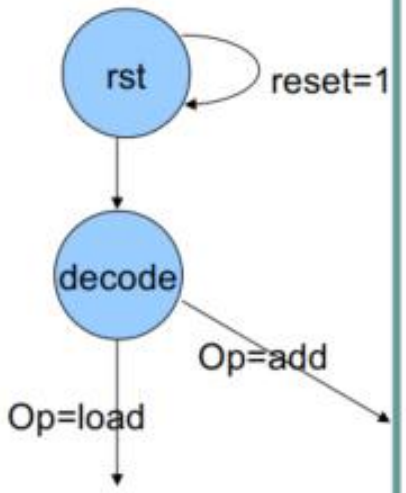
Verilog/VHDL code template for implementing the Controller's Finite State Machine (FSM)

- 1. State Encoding (parameter)
  - Reset: state 0 / Decode: state 1
- 2. Clock Trigger
  - always @(negedge clk)
  - logic block executes on the *falling edge* (negedge) of the clock signal
  - Timing ensures that control signals are updated and stable before the next rising edge, which is when the datapath registers (like PC or pipeline registers) typically capture data.

State Machine Implementation

```
parameter [3:0]
        RESET=0,DECODE=1;

always @(negedge clk)
        if(rst)
                begin
                state = RESET;
                //deactivate all control signals
                end
        else
                begin
                case(state)
                        RESET:  begin
                                state=DECODE;
                        end
                        DECODE: begin
                                if(opcode=ADD)
                                ...
                        end
                        default:
                                state=RESET;
                endcase
                end
```



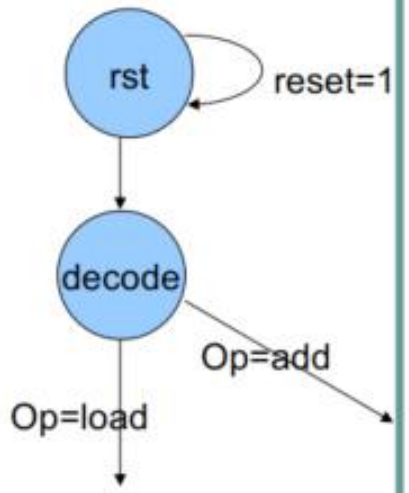
Slide 51 [1] State Machine Implementation

- 3. Reset Logic (if(rst))
  - if(rst) begin state = RESET; … End
  - This is a high-priority check. If the hardware rst signal is high (1), the FSM immediately forces the state to RESET
  - *Control Signals:* write code here to force signals like *wr_en* (Write Enable) to 0 so the CPU does not accidentally write to memory while resetting.
- 4. State Transitions (case(state))
  - RESET Case: after a reset, the CPU moves directly to DECODE to start processing instructions.
  - You must modify this section to check if the user pressed *Reset & Load* (loading code) or *Reset & Execute* (running code) and set the Program Counter (PC) accordingly.

State Machine Implementation

```
parameter [3:0]
        RESET=0,DECODE=1;

always @(negedge clk)
        if(rst)
                begin
                state = RESET;
                //deactivate all control signals
                end
        else
                begin
                case(state)
                        RESET:  begin
                                state=DECODE;
                        end
                        DECODE: begin
                                if(opcode=ADD)
                                …
                        end
                        default:
                                state=RESET;
                endcase
                end
```



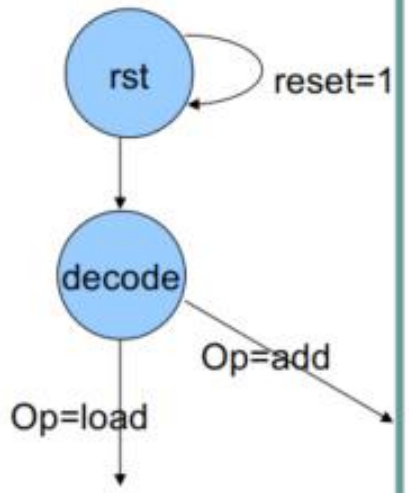
Slide 51 [1] State Machine Implementation

- 3. Reset Logic (if(rst))
  - if(rst) begin state = RESET; … End
  - This is a high-priority check. If the hardware rst signal is high (1), the FSM immediately forces the state to RESET
  - *Control Signals:* write code here to force signals like *wr_en* (Write Enable) to 0 so the CPU does not accidentally write to memory while resetting.
- 4. State Transitions (case(state))
  - RESET Case: after a reset, the CPU moves directly to DECODE to start processing instructions.
  - You must modify this section to check if the user pressed *Reset & Load* (loading code) or *Reset & Execute* (running code) and set the Program Counter (PC) accordingly.

State Machine Implementation

```
parameter [3:0]
        RESET=0,DECODE=1;

always @(negedge clk)
        if(rst)
                begin
                state = RESET;
                //deactivate all control signals
                end
        else
                begin
                case(state)
                        RESET:  begin
                                state=DECODE;
                        end
                        DECODE: begin
                                if(opcode=ADD)
                                …
                        end
                        default:
                                state=RESET;
                endcase
                end
```



The reset state needs to be modified to account for the two different resets as per slides 2 & 3

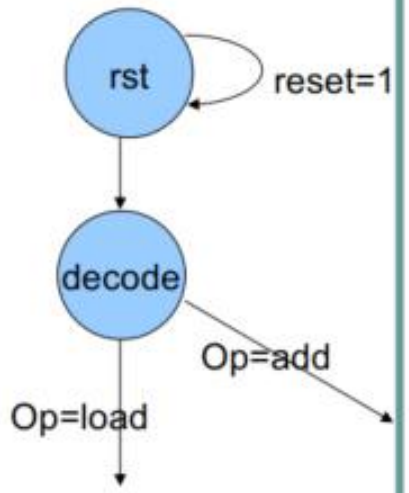
Slide 51 [1] State Machine Implementation

- 4. State Transitions (case(state))
  - DECODE Case:
    - "brain" of the cycle. It checks the opcode (e.g., if(opcode == ADD)) to decide which control signals to activate and which state to go to next.
  - default Case:
    - If the FSM enters an unknown state (due to a glitch or error), this forces the CPU back to the start (RESET) to recover safely
- This is a template: need flesh out the DECODE block to handle all your instructions and modify the RESET block to handle the two distinct boot modes required by the project.

State Machine Implementation

```
parameter [3:0]
        RESET=0,DECODE=1;

always @(negedge clk)
        if(rst)
                begin
                state = RESET;
                //deactivate all control signals
                end
        else
                begin
                case(state)
                        RESET:  begin
                                state=DECODE;
                        end
                        DECODE: begin
                                if(opcode=ADD)
                                ...
                        end
                        default:
                                state=RESET;
                endcase
                end
```



Slide 52 [1]: Implementation strategy

1. First Design Datapath
   - Start by building physical ‘muscles’ of the CPU before building the brain
   - code the individual Verilog modules for components like the *ALU (Arithmetic Logic Unit), Register File, Program Counter (PC), and Memory*
   - You cannot design a controller to drive an ALU if you don't yet know what control signals that ALU needs (e.g., does it need a 2-bit signal or a 4-bit signal to switch between ADD and SUB?)
2. Design Controller Based on Data-Path
   - After finishing datapath, have a list of all the control inputs it needs *(e.g., alu_mode, reg_write_en, mem_read_en)*. The Controller is designed specifically to generate these signals.
   - write the *Finite State Machine (FSM)* or decoder logic that looks at an instruction's Op-Code and activates the specific wires connected to your datapath components.
   - Change the Datapath (e.g., add a new register bank), you must update the Controller to match.

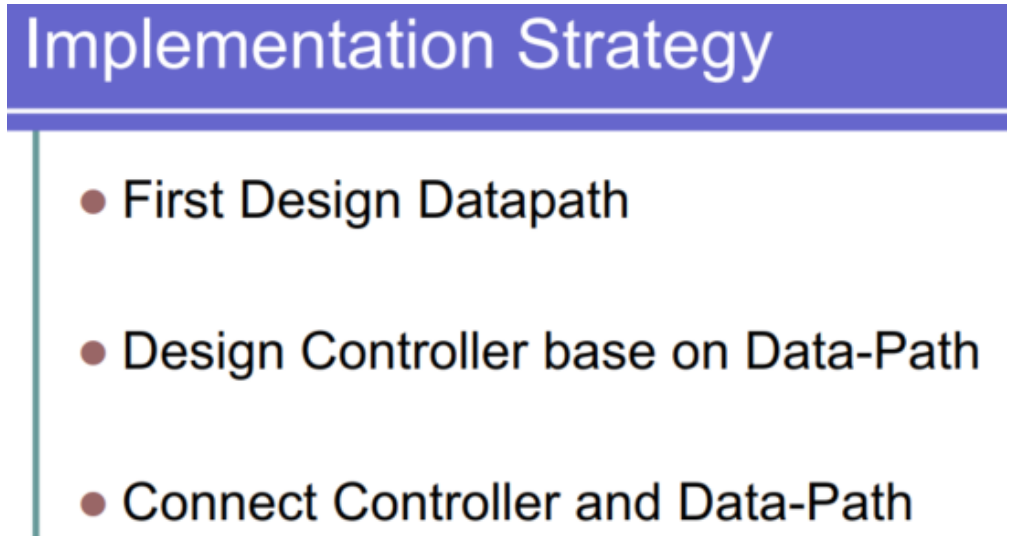
Implementation Strategy

- First Design Datapath
- Design Controller base on Data-Path
- Connect Controller and Data-Path

Slide 52 [1]: Implementation strategy

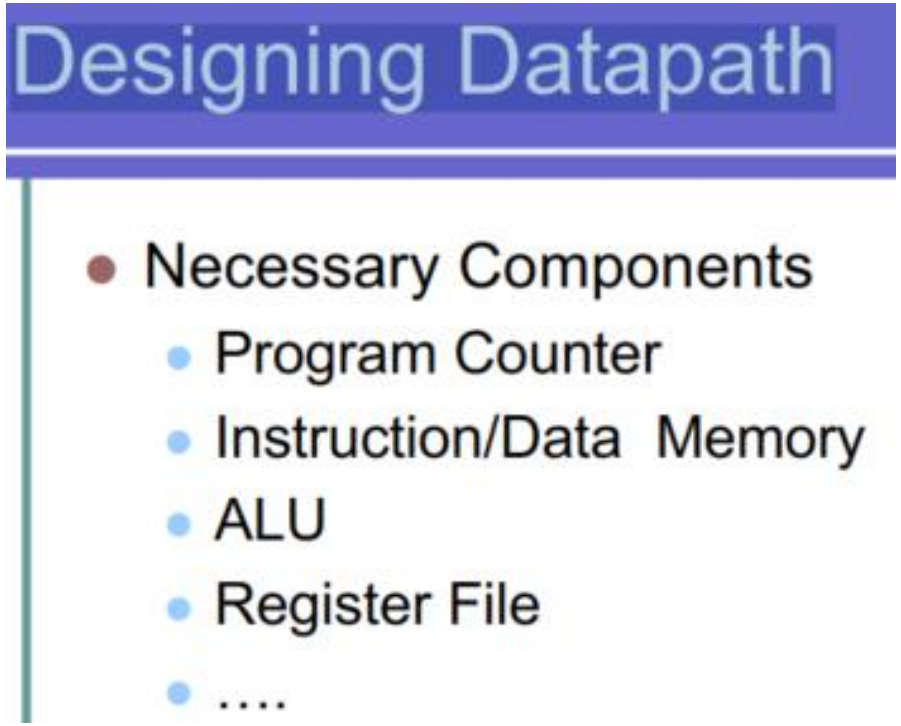
3. Connect Controller and Data-Path

- Integration phase where you create the *Top-Level Module* (often called CPU.v or System.v).
- Instantiate both the *Datapath* module and the *Controller* module inside a parent module and connect them with wires
  - Outputs from Controller → Inputs to Datapath (e.g., "Write to Register 3").
  - Outputs from Datapath → Inputs to Controller (e.g., "The ALU result is Zero,"



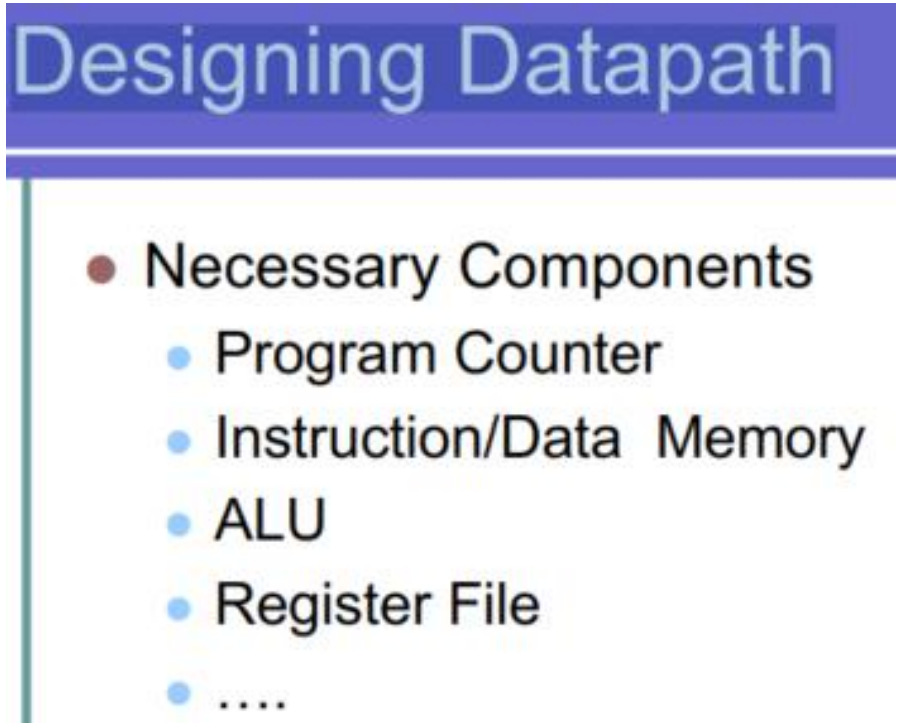
Slide 53 [1] Designing Datapath

- 1. Program Counter (PC):
  - A special register that holds the memory address of the next instruction to be executed. points to the Instruction Memory to fetch code. Should support resetting to 0x0000 (for Reset & Execute) or 0x0002 (for Reset & Load).
- 2. Instruction/Data Memory:
  - storage unit for your program code and variables. hold instructions. holds data variables.
  - uses a *Dual Ported RAM* (Harvard Architecture) to ensure instruction and data traffic are separated, allowing the CPU to read an instruction and access data simultaneously



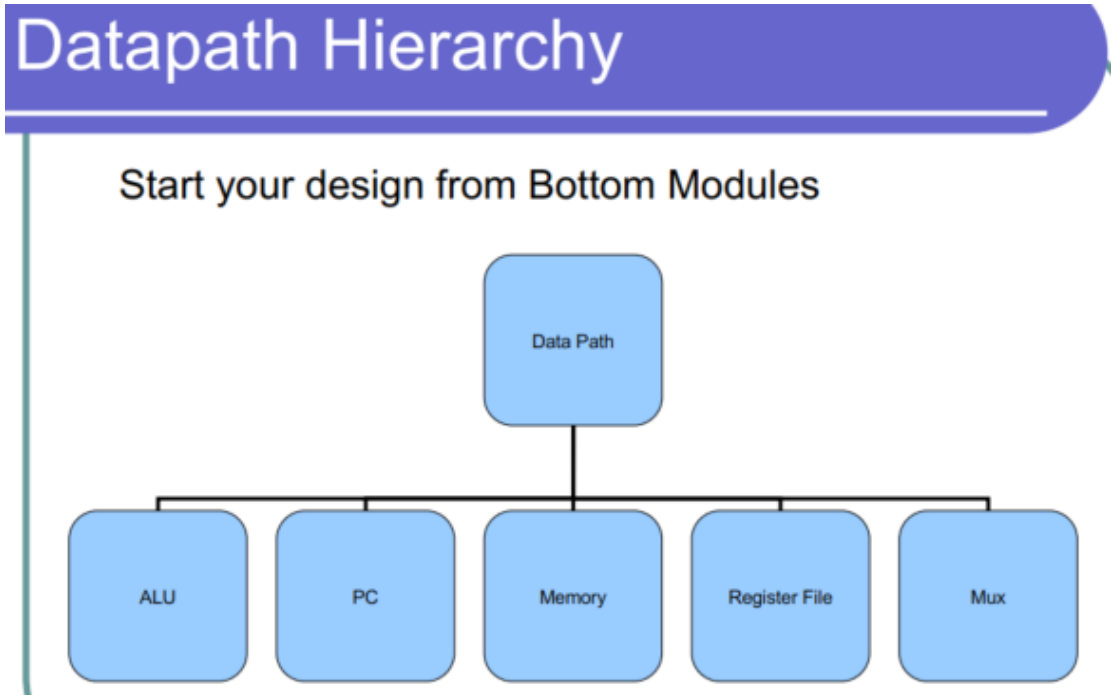
Slide 53 [1] Designing Datapath

- 3. ALU (Arithmetic Logic Unit):
  - computational engine of the CPU.computational engine of the CPU. Executes arithmetic instructions (like ADD, SUB, MUL) and logic operations (like NAND, SHL). It takes two operands from the Register File and produces a result
- 4. Register File
  - A small, fast storage bank inside the CPU.
  - registers *r0 through r7*. has two read ports (to read op1 and op2 for the ALU) and one write port (to save the result). *r7* special role as the Link Register for subroutines.
- Other components:
  - *Mux (Multiplexers):* To select between different data sources (e.g., choosing whether the ALU input comes from a register or an immediate value).
  - *Pipeline Registers:* Registers like IF/ID, ID/EX, EX/MEM, and MEM/WB used to separate the 5 stages of the pipeline .



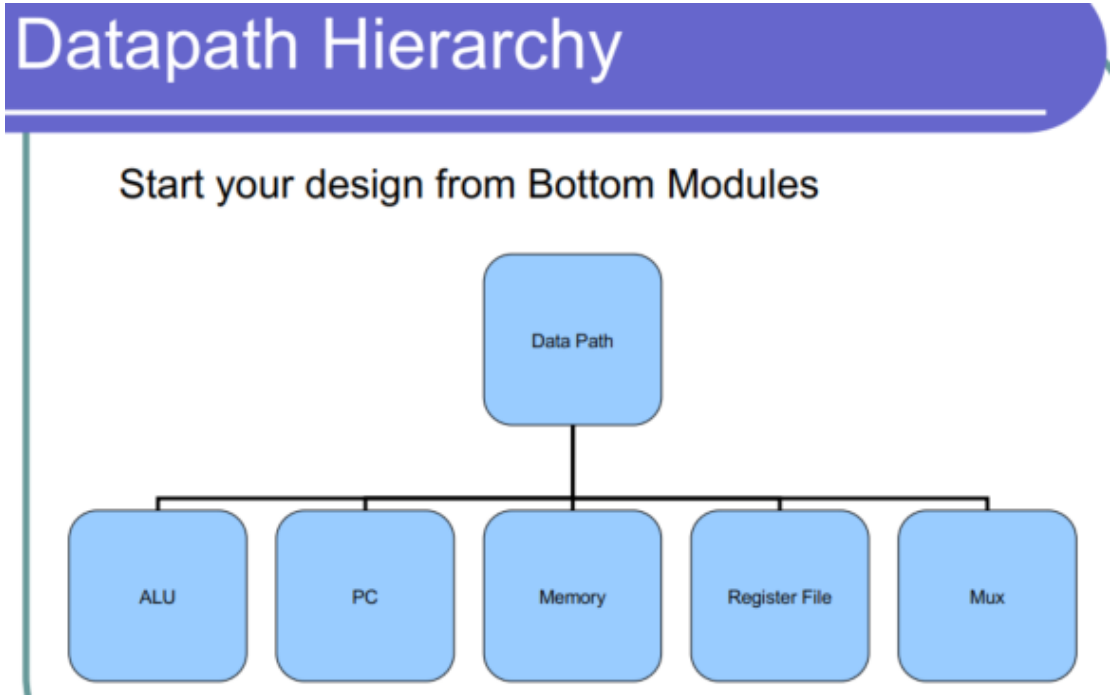
Slide 54 [1] Datapath Hierarchy

- Start your design from Bottom Modules
  - Don’t write the top-level Data Path module first. Instead, write and test the smallest, independent components *(like the ALU or Mux)* individually. Only once those are working should you create the container that holds them.
- Top-Level Module: Data Path
  - *instantiate* all the lower-level modules and connect them with wires. For example, it connects the output wire of the PC to the address input wire of the Memory.



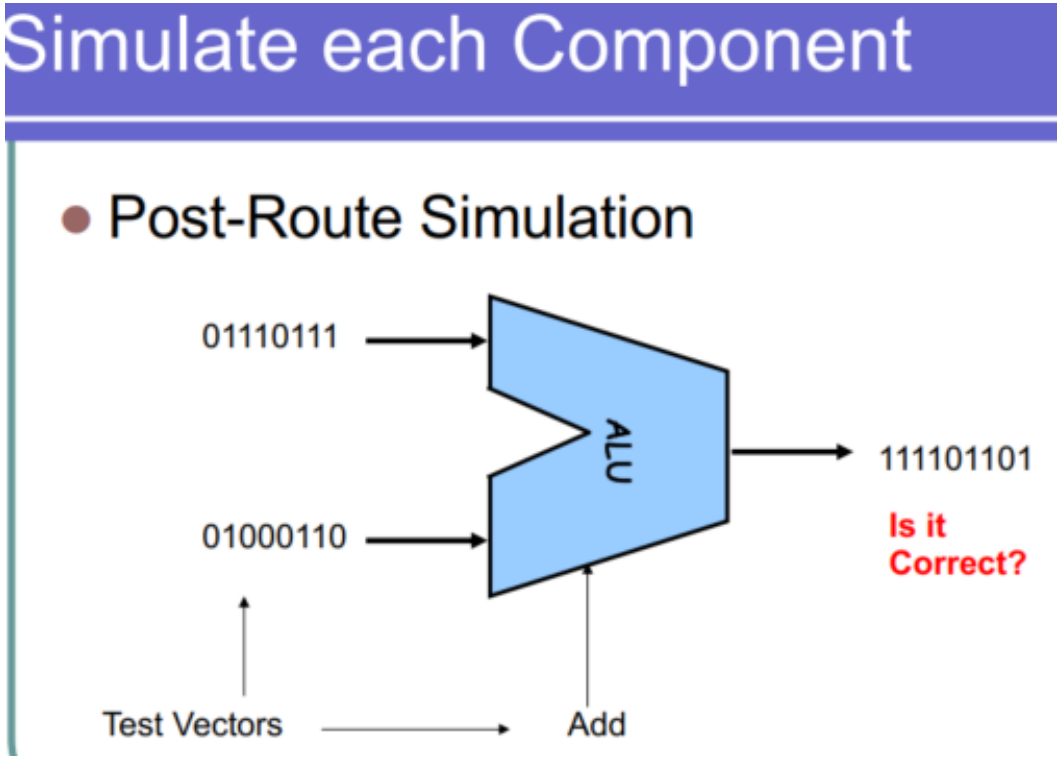
Slide 54 [1] Datapath Hierarchy

- Bottom-Level Modules (The Sub-Components)
  - ALU: performs the actual calculations (ADD, SUB, NAND, etc.) requested by the instruction
  - PC (Program Counter): maintains the current address. It needs logic to increment (PC + 2) for normal execution or load a new value (Branch Target) during jumps.
  - Memory: instantiate the *Dual Ported RAM* macro provided by Xilinx to separate instruction fetch from data access.
  - Register File**:** 8 general-purpose registers It handles reading two values at once (for operations like ADD r1, r2, r3) and writing one result back
  - *Mux (Multiplexer) :* select between different data sources. decide if the ALU input comes from a Register (e.g., r2) or from an Immediate value (e.g., 5 in ADD r1, #5).



Slide 55 [1] Simulate each Component

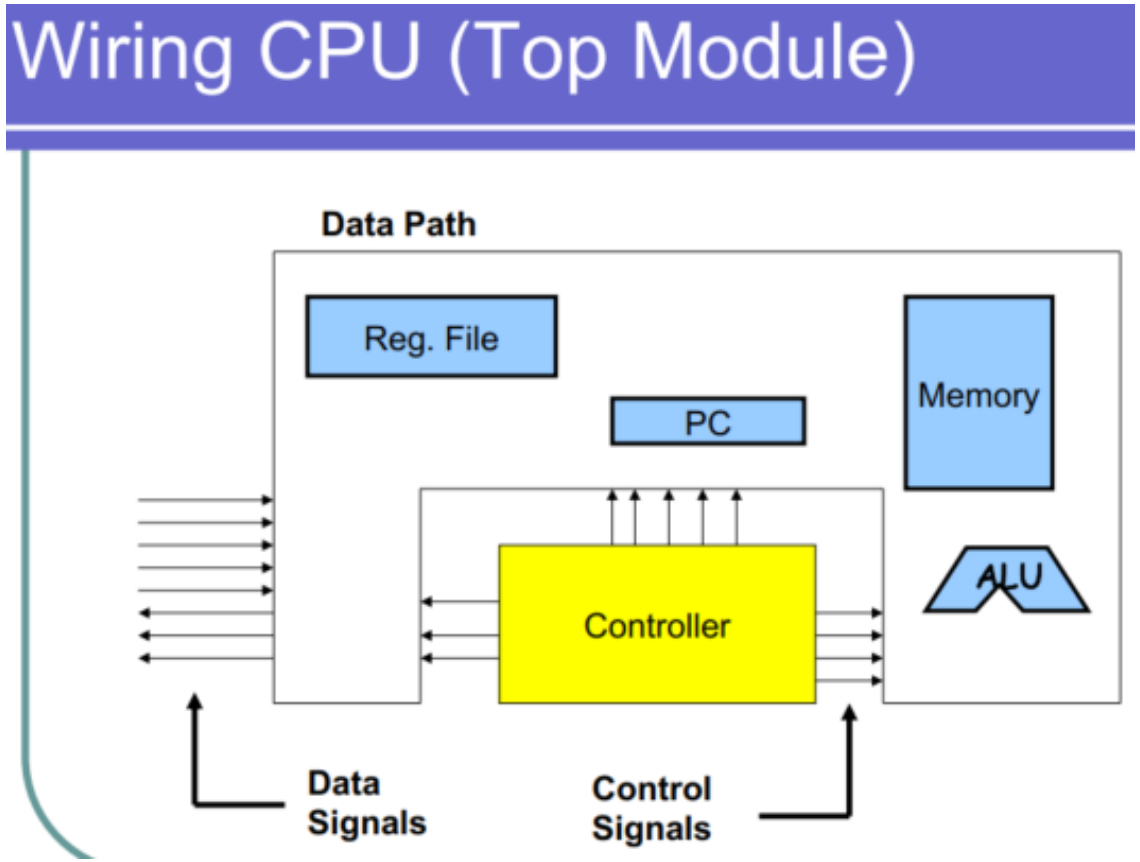
- Unit Testing: before you assemble the entire CPU, you must simulate and verify every single individual module (like the ALU) to ensure it behaves exactly as expected.
- *Post-Route* simulation runs on the design *after* it has been mapped to the physical FPGA layout. It includes real-world timing delays and physical wire properties
  - It ensures that your design not only logically adds numbers correctly but also does so fast enough to meet the timing constraints of the hardware



Math Check: 01110111 + 01000110 = 10111101
(Note: The slide's example output 111101101 is actually a 9-bit result, implying it includes the Carry Out bit)

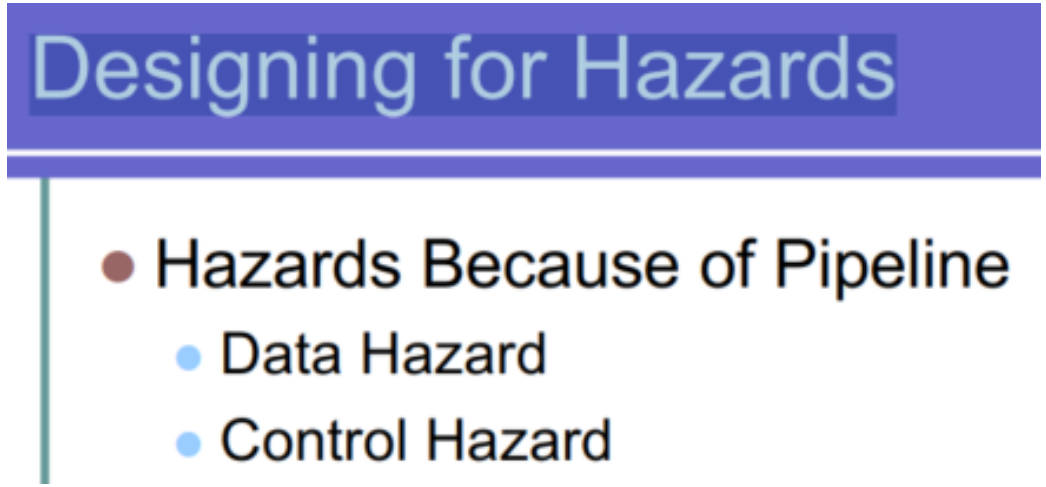
Slide 56 [1] Wiring CPU (Top Module)

- 1. Data Path large block represents the collection of all your operational hardware modules.
  - holds the state and performs the actual data processing. It receives control signals to know *what* to do and outputs status flags
- 2. Controller: It generates to drive the components.
  - Finite State Machine (FSM). reads the current instruction's Op-code and orchestrates the Datapath.
  - Receives the *Instruction* (from Memory/IR) and *Status Flags* (from the ALU) to make decisions.
- 3. Signals
  - Control Signals
  - Data Signals:



Slide 57 [1] Designing for Hazards

- 1. Hazards Because of Pipeline
  - A hazard occurs when the *next instruction in the pipeline cannot execute in the following clock cycle because it depends on an event or data that hasn't happened yet*
  - In your 5-stage design, while Inst 1 is in the "Write Back" stage, Inst 4 might be in the "Decode" stage. If Inst 4 needs the result from Inst 1, it might try to read it *before* Instruction 1 has actually written it.



Slide 57 [1] Designing for Hazards

- *2.* Data hazard: This occurs when an instruction depends on the *data result* of a previous instruction that is still in the pipeline.
  - E.g. *Inst A:* ADD r1, r2, r3 & *Inst B*: SUB r4, r1, r5
  - In a pipeline, Instruction B fetches r1 in the *Decode* stage. However, Instruction A doesn't write the new value into r1 until the *Write Back* stage (3 cycles later). If you don't "design for hazards," Instruction B will read the *old* value of r1, leading to a wrong calculation.
  - Solution: likely need to implement *Forwarding* (passing the result directly from ALU to ALU) or *Stalling* (pausing the CPU).

Designing for Hazards

- Hazards Because of Pipeline
  - Data Hazard
  - Control Hazard

Slide 57 [1] Designing for Hazards

- 3. Control Hazard:This occurs due to *Branching* instructions (Format B instructions like BRR or BR).
  - The CPU defaults to fetching the next instruction (PC + 2). However, a Branch instruction might change the PC to a completely different address.
  - The decision to branch is made in the *Execute* stage (when the ALU checks flags like Zero or Negative). By the time this decision is made, the pipeline has already fetched the next 2 instructions incorrectly.
  - *Solution:* You often need to "Flush" (delete) the incorrectly fetched instructions or use "Branch Prediction."

Designing for Hazards

- Hazards Because of Pipeline
  - Data Hazard
  - Control Hazard

Slide 58 [1] Simulate and Implement

- 1. Simulating Complete CPU
  - Unlike the component simulations (slide 55) where you tested just an ALU or Register File, this step tests the *entire processor* working together.
  - write a testbench that acts as the "Instruction Memory." You load a sequence of binary instructions (a small program) into the CPU's memory model and run the simulation.
  - Outcomes:
    - Does the PC increment correctly?
    - Do Hazards (from slide 57) cause errors, or does the logic handle them?
    - Do instructions like ADD, LOAD, and BRANCH execute in the correct order and produce the right values in the Register File?

Simulate and Implement

- Simulating Complete CPU
- Implementation (Pin Assignment)

Slide 58 [1] Simulate and Implement

- 2. Implementation (Pin Assignment)
  - Your Verilog/VHDL code has inputs called *clk, reset, in_port and outputs called out_port*. The FPGA chip doesn't know which physical metal pins these names correspond to.
  - create a *Constraints File (.xdc)*. This file maps your signal names to specific pin numbers on the FPGA chip.
    - *E.g* Map clk to pin W5 (the onboard oscillator).
    - Map reset to the center button.
    - Map out_port bits to the 16 LEDs on the board

Simulate and Implement

- Simulating Complete CPU
- Implementation (Pin Assignment)